

AC696N_soundbox_SDK_介绍

AC696N_soundbox_SDK_介绍 用户手册

Rev 2.0 —— 2020 年 08 月 20 日

This translated version is for reference only, and the English version shall prevail in case of any discrepancy between the translated and English versions.

版权所有 2020 杰理科技有限公司未经许可，禁止转载

目录

Chapter 1	SDK 功能说明.....	5
1.1	编写目的.....	5
1.2	模式介绍.....	5
1.3	细分功能介绍.....	6
1.4	是否使用配置文件的参数配置.....	9
1.5	长按复位设置.....	10
1.6	功能选择注意事项.....	11
Chapter 2	VM 接口的使用.....	12
2.1	编写目的.....	12
2.2	详细说明.....	12
Chapter 3	APP 模式管理.....	15
3.1	编写目的.....	15
3.2	模式任务.....	15
3.3	模式流程的详细介绍（以上电进入 powerno 模式为例）.....	17
3.4	APP 模式管理函数接口介绍.....	20
3.5	增加 APP 模式例子（以 music 为例）.....	23
Chapter 4	APP 消息管理.....	27
4.1	系统事件管理.....	27
4.2	提供的 APP 消息处理函数说明.....	28
Chapter 5	杰理之家 APP.....	32
5.1	编写目的.....	32
5.2	APP 介绍.....	32
5.3	文件目录说明.....	33
5.4	功能开关、配置.....	34
5.5	命令、数据 API 介绍.....	35
5.6	功能配置.....	40
Chapter 6	APP 升级固件说明.....	44
6.1	编写目的.....	44
6.2	使用 sdk 自带协议进行 OTA 升级.....	44
6.3	客户自带协议进行 OTA 升级.....	46
6.4	升级流程图.....	48

Chapter 7 SD、USB 复用 IO 介绍.....	49
7.1 编写目的.....	49
7.2 sd 复用 api 说明.....	49
7.3 sd cmd 复用 linein ad 检测.....	50
7.4 sd clk 复用 iokey 检测.....	51
7.5 sd data 复用 usb dm.....	52
Chapter 8 Audio 模块使用.....	54
8.1 编写目的.....	54
8.2 详细说明.....	54
8.3 MIC 和 LINEIN 增益调节.....	54
Chapter 9 客户常见问题反馈解答.....	56
Q1:	56
Q2:	56

修改日志

版本	日期	描述
2.0	2020 / 8 / 20	AC696N 用户手册
更新:	● 建立初始版本 ● 定义文档格	

Chapter 1 SDK 功能说明

1.1 编写目的

该文档主要描述 AC696x_SDK 整体功能介绍，开发人员可通过此文档快速了解 sdk 功能

1.2 模式介绍

1. AC696x_SDK 支持模式有：

- (1) 蓝牙模式
- (2) 音乐模式
- (3) FM 模式
- (4) PC 模式
- (5) LINEIN 模式
- (6) 录音模式（暂支持单独 mic 录音）
- (7) RTC 模式（非真正的硬件 RTC，上电会重置）

2. 支持模式开关

```
21 #define NO_CONFIG_PORT (1)
22
23 /******* app 配置 *****/
24 //
25 /*******
26 #define TCFG_APP_BT_EN 1
27 #define TCFG_APP_MUSIC_EN 1
28 #define TCFG_APP_LINEIN_EN 1
29 #define TCFG_APP_FM_EN 1
30 #define TCFG_APP_PC_EN 1
31 #define TCFG_APP_RTC_EN 0
32 #define TCFG_APP_RECORD_EN 1
33 #define TCFG_APP_SPDIF_EN 0
34 /*******
35 //
36 /******* PCM_DEBUG调试配置 *****/
37
38 // #define AUDIO_PCM_DEBUG //PCM串口调试，写卡通话数据
39
40 /*******
41 //
42 /******* UART配置 *****/
43 #define TCFG_UART0_ENABLE ENABLE_THIS_MODULE //串口打印模块使能
44 #define TCFG_UART0_RX_PORT NO_CONFIG_PORT //串口接收脚配置（用于
45 #define TCFG_UART0_TX_PORT IO_PORTA_05 //串口发送脚配置
46 #define TCFG_UART0_BAUDRATE 1000000 //串口波特率配置
47
48 /*******
49 //
50 /******* IIC配置 *****/
51
NORMAL master apps/soundbox/board/br25/board_ac696x_demo/board_ac696x_demo_cfg.h
ac696x_demo/board_ac696x_demo_cfg.h" 794L, 40533C
```

1.3 细分功能介绍

蓝牙	高级音频	常规操作
		音量同步
		歌词 id 信息
		歌曲时间
	通话	回拨
		接听拒接
		声卡切换
		三方通话
	spp	
	hid	
	ble	弹窗、透传
		可支持开发 app
	tws 对箱（暂不支持）	蓝牙播歌通话（通话主机 dac 出声音）
		解码（mp3、wma、wav）
		linein、pc、fm 等 pcm
		混响（内置 fm 不支持）
	后台	fm、蓝牙不能共存
解码	格式	CVSD\MSBC\SBC\AAC\G729\MP3\WMA\FLAC\APE\M4A\AMR\DTS\G726\MIDI
	加密播放	
音频播放	按路径播放	
	音频位置信息	文件夹总数、文件数、当前文件在当前文件夹序
	播放模式	单曲、循环、随机……
	文件系统	fat12、fat16、fat32、extfat

编码	格式	CVSD、MSBC\G726\MP3\ADPCM\SBC
音源输入	iis	暂不支持
	hdmi	暂不支持
	spdif	暂不支持
	linein	模拟、数字
	pc	
音效	混响	
	eq、drc\在线调试	支持 20 段
	音效、在线调试	魔音、啸叫抑制、变速 变调、高低音、电音……
FM	内置 fm 收音	立体声、单声道、50k、100k 步进搜台
rtc	时钟、闹钟	非真正的硬件时钟，重新上电会重置，暂不能进低功耗
外设	双卡	分时复用
	usb	pc (massstorage、speaker、mic、hid)
	flash	支持分区、文件系统、pc 出盘符、提示音
提示音	格式	MP3、wtg\msbc\sbc\sin\mty
按键	iokey	
	adkey	
	irkey	红外
	rdeckey	旋码器
	slidekey	滑动电阻按键

audio	out	dac
		Iis （暂不支持）
		fm
		hdmi （暂不支持）
		sdpif （暂不支持）
		bt
	channel	MONO_L\左声道
		MONO_R\右声道
		LR\立体声
		MONO_LR_DIFF\单声道差分输出
DUAL_LR_DIFF\双声道差分		
显示	led	
	断码 lcd	
升级	U 盘	支持 uboot ui、io 保持、ufw 文件只识别后缀
	sd 卡	
	串口	
	测试盒	
	烧写器	
	内置充电	
	系统时钟	最高 192M
	不可屏蔽中断有 些级	优先级 6 以上不开屏蔽
	adc	只支持 1 路 ad

1.4 是否使用配置文件的参数配置

默认 sdk 是从配置文件读取配置来设置灯状态和提示音，如果不想从配置文件读取配置，只需修改

```
#define USE_CONFIG_BIN_FILE 0

#define USE_CONFIG_STATUS_SETTING 1
#define USE_CONFIG_AUDIO_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_CHARGE_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_KEY_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_PWMLED_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_MIC_TYPE_SETTING USE_CONFIG_BIN_FILE
```

成

```
#define USE_CONFIG_BIN_FILE 0

#define USE_CONFIG_STATUS_SETTING 0
#define USE_CONFIG_AUDIO_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_CHARGE_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_KEY_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_PWMLED_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_MIC_TYPE_SETTING USE_CONFIG_BIN_FILE
```

即可，然后默认配置在每个 board.c 里面设置

```
STATUS_CONFIG status_config = {
    //灯状态设置
    led = {
        .charge_start = PWM_LED1_ON,
        .charge_full = PWM_LED0_ON,
        .power_on = PWM_LED0_ON,
        .power_off = PWM_LED1_FLASH_THREE,
        .lowpower = PWM_LED1_SLOW_FLASH,
        .max_vol = PWM_LED_NULL,
        .phone_in = PWM_LED_NULL,
        .phone_out = PWM_LED_NULL,
        .phone_activ = PWM_LED_NULL,
        .bt_init_ok = PWM_LED0_LED1_FAST_FLASH,
        .bt_connect_ok = PWM_LED0_SLOW_FLASH,
        .bt_disconnect = PWM_LED0_LED1_FAST_FLASH,
    },
    //提示音设置
    tone = {
        .charge_start = IDEX_TONE_NONE,
        .charge_full = IDEX_TONE_NONE,
        .power_on = IDEX_TONE_POWER_ON,
        .power_off = IDEX_TONE_POWER_OFF,
        .lowpower = IDEX_TONE_LOW_POWER,
        .max_vol = IDEX_TONE_MAX_VOL,
        .phone_in = IDEX_TONE_NONE,
        .phone_out = IDEX_TONE_NONE,
        .phone_activ = IDEX_TONE_NONE,
        .bt_init_ok = IDEX_TONE_BT_MODE,
        .bt_connect_ok = IDEX_TONE_CONN,
        .bt_disconnect = IDEX_TONE_DISCONN,
    }
};
```

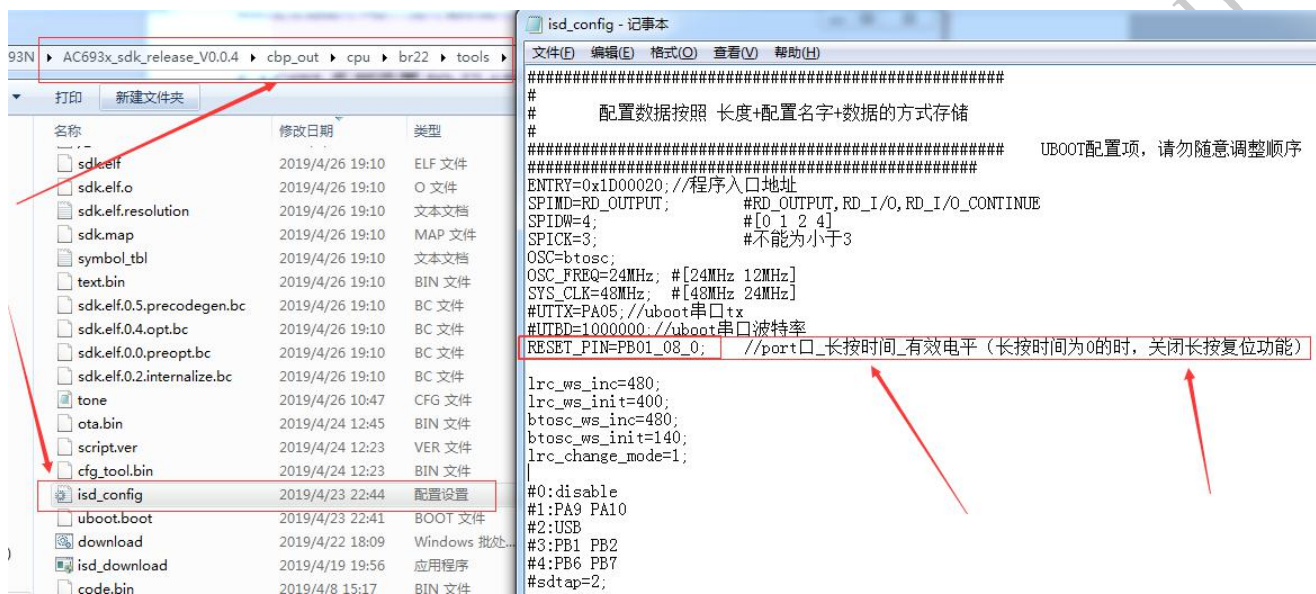
注意：如果想生成的 fw 文件能单独配置某些选项，要把对应的宏打开，如果想全部打开则设置

```
#define USE_CONFIG_BIN_FILE 1
```

即可

1.5 长按复位设置

设置在如下路径...\cbp_out\cpu\br25\tools 的 isd_config.ini 文件设置



长按时间有 00、04、08 可选, 其他值默认选 04

图 2

1.6 功能选择注意事项

AC696x_SDK 功能比较全面独立，开发人员可以按照需求组合功能，但是仍有一些功能，因为资源和速度的限制，例如蓝牙后台不支持 FM，混响不支持通话回声等，SDK 做了一些编译限制，例如：

```

//*****编译警告*****//
//*****编译警告*****//
//*****编译警告*****//
#if (((TCFG_ONLINE_TX_PORT == IO_PORT_DP) && TCFG_ONLINE_ENABLE)) && (TCFG_PC_ENABLE || TCFG_UDISK_ENABLE))
#error "eq online adjust enable, please close usb marco !!!"
#endif// ((TRANS_DATA_EN || TCFG_ONLINE_ENABLE) && (TCFG_PC_ENABLE || TCFG_UDISK_ENABLE))

#if TCFG_UI_ENABLE
#if ((TCFG_SPI_LCD_ENABLE && TCFG_CODE_FLASH_ENABLE) && (TCFG_FLASH_DEV_SPI_HW_NUM == TCFG_TFT_LCD_DEV_SPI_HW_NUM))
#error "flash spi port == lcd spi port, please close one !!!"
#endif//((TCFG_SPI_LCD_ENABLE && TCFG_CODE_FLASH_ENABLE) && (TCFG_FLASH_DEV_SPI_HW_NUM == TCFG_TFT_LCD_DEV_SPI_HW_NUM))
#endif//TCFG_UI_ENABLE

#if(TRANS_DATA_EN && DUEROS_DMA_EN)
#error "they can not enable at the same time,just select one!!!"
#endif//((TRANS_DATA_EN && DUEROS_DMA_EN))

#if (TCFG_DEC2TWS_ENABLE && (TCFG_APP_RECORD_EN || TCFG_APP_RTC_EN ||TCFG_DRC_ENABLE))
#error "对箱支持资源转发，请关闭录音等功能 !!!"
#endif// ((TCFG_DEC2TWS_ENABLE && (TCFG_APP_RECORD_EN || TCFG_APP_RTC_EN ||TCFG_DRC_ENABLE))

#if ((TCFG_NORFLASH_DEV_ENABLE || TCFG_NOR_FS_ENABLE) && TCFG_UI_ENABLE)
#error "引脚复用问题，使用norflash需要关闭UI !!!"
#endif

#if CONFIG_DOUBLE_BANK_ENABLE
#error "not support!!!"
#endif//
//<<<<<<所有宏定义不要在编译警告后面定义!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
//<<<<<<所有宏定义不要在编译警告后面定义!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

//*****配置结束*****//
//*****配置结束*****//
//*****配置结束*****//

#endif //CONFIG_BOARD_AC693X_DEMO
RMAL master apps/soundbox/board/br25/board_ac696x_smartbox/board_ac696x_smartbox.h cpp utf-8
```

Chapter 2 VM 接口的使用

2.1 编写目的

该文档主要描述 AC696x_SDK VM 使用方法及开发中注意的一些问题，为用户进行二次开发提供参考

2.2 详细说明

Vm 读写接口：

读接口：

```
int syscfg_read(u16 item_id, void *buf, u16 len);
```

写接口：

```
int syscfg_write(u16 item_id, void *buf, u16 len);
```

参数说明：

item_id: vm id 范围 0 ~ 50

*buf: 读写指针

Len: 长度

比如保存 USB MIC GAIN 和读 USB MIC GAIN 的 vm 操作：

1、确定 vm id 号

```

3
4 //-----//
5 //                                     与APP CASE相关配置项[1 ~ 60]                                     //
6 //-----//
7 // #define VM_FM_EMITTER_FREQ          1
8 // #define VM_FM_EMITTER_DIG_VOL      2
9 // #define VM_MUSIC_LAST_DEV          3
10 // #define VM_WAKEUP_PND              4
11 #define VM_USB_MIC_GAIN              5
12 #define VM_ALARM_0                   6
13 #define VM_ALARM_1                   7
14 #define VM_ALARM_2                   8
15 #define VM_ALARM_3                   9
16 #define VM_ALARM_4                   10
17 #define VM_ALARM_MASK                11
18 #define VM_ALARM_NAME_0              12
19 #define VM_ALARM_NAME_1              13
20 #define VM_ALARM_NAME_2              14
21 #define VM_ALARM_NAME_3              15
22 #define VM_ALARM_NAME_4              16
23 #define VM_FM_INFO                   17
24 // #define VM_RTC_TRIM                18
25 #define CFG_RCSP_ADV_HIGH_LOW_VOL    19
26 #define CFG_RCSP_ADV_EQ_MODE_SETTING 20
27 #define CFG_RCSP_ADV_EQ_DATA_SETTING 21
28 #define ADV_SEQ_RAND                 22
29 #define CFG_RCSP_ADV_TIME_STAMP      23
30 #define CFG_RCSP_ADV_WORK_SETTING     24
31 #define CFG_RCSP_ADV_MIC_SETTING     25
32 #define CFG_RCSP_ADV_LED_SETTING     26
33 #define CFG_RCSP_ADV_KEY_SETTING     27
34 #define CFG_RCSP_ADV_VOL_SETTING     28
35 // #define CFG_FLASH_BREAKPOINT       29
36 // #define CFG_USB_BREAKPOINT         30
37 // #define CFG_SD0_BREAKPOINT         31
38 // #define CFG_SD1_BREAKPOINT         32
39 // #define CFG_MUSIC_DEVICE           33
40 // #define CFG_FM_RECEIVER_INFO       34
41 // #define CFG_FM_TRANSMIT_INFO       35
42 // #define CFG_AAP_MODE_INFO          36
43 // #define CFG_UI_SYS_INFO            37
44
45 NORMAL master apps/soundbox/include/user_cfg_id.h
46 c|293| <<<unknown>>> ret = syscfg_read(VM_USB_MIC_GAIN, &usb_mic_gain, 1);

```

2、写 USB MIC GAIN

```

1 usb_mic_gain_save_cnt = 0;
2 local_irq_enable();
3 printf("USB_GAIN_SAVE\n");
4 syscfg_write(VM_USB_MIC_GAIN, &app_var.usb_mic_gain, 1);
5 return;
6 }
7 local_irq_enable();

```

3、读 USB MIC GAIN

```

:  /// pc usb mic gain
:  s8 usb_mic_gain = -1;
:  ret = syscfg_read(VM_USB_MIC_GAIN, &usb_mic_gain, 1);
:  if (ret < 0) {
:      usb_mic_gain = 7;
:  }
:  app_var.usb_mic_gain = usb_mic_gain;
:  log_info("usb_mic_gain: %d\n", app_var.usb_mic_gain);
:
:  s8 default_volume = -1;
:  s8 music_volume = 1;

```

Chapter 3 APP 模式管理

3.1 编写目的

该文档主要描述 AC696x_SDK APP 模式使用方法及开发中注意的一些问题，为用户进行二次开发提供参考

3.2 模式任务

1. 模式任务 app_core 创建，所有的 APP 模式在此任务里执行处理

```
int main()
{
    wdt_close();

    os_init();

    setup_arch();

    board_early_init();

    task_create(app_task_handler, NULL, "app_core");

    os_start();

    local_irq_enable();

    while (1) {
        asm("idle");
    }

    return 0;
}
```

RMAL master apps/soundbox/common/init.c

2. APP 模式处理函数 app_task_loop()



```
void app_task_loop()
{
    while (1) {
        switch (app_curr_task) {
            case APP_POWERON_TASK:
                log_info("APP_POWERON_TASK \n");
                app_poweron_task();
                break;
            case APP_POWEROFF_TASK:
                log_info("APP_POWEROFF_TASK \n");
                app_poweroff_task();
                break;
            case APP_BT_TASK:
                log_info("APP_BT_TASK \n");
                app_bt_task();
                break;
            case APP_MUSIC_TASK:
                log_info("APP_MUSIC_TASK \n");
                app_music_task();
                break;
            case APP_FM_TASK:
                log_info("APP_FM_TASK \n");
                app_fm_task();
                break;
            case APP_RECORD_TASK:
                log_info("APP_RECORD_TASK \n");
                app_record_task();
                break;
            case APP_LINEIN_TASK:
                log_info("APP_LINEIN_TASK \n");
                app_linein_task();
                break;
            case APP_RTC_TASK:
                log_info("APP_RTC_TASK \n");
                app_rtc_task();
                break;
            case APP_PC_TASK:
                log_info("APP_PC_TASK \n");
                app_pc_task();
                break;
            case APP_SPDIF_TASK:
                log_info("APP_SPDIF_TASK \n");
                app_spdif_task();
                break;
            case APP_IDLE_TASK:
                log_info("APP_IDLE_TASK \n");
                app_idle_task();
                break;
        }
    }
}
```

3. APP 模式值

```
enum {  
    APP_POWERON_TASK    = 1,  
    APP_POWEROFF_TASK   = 2,  
    APP_BT_TASK          = 3,  
    APP_MUSIC_TASK       = 4,  
    APP_FM_TASK          = 5,  
    APP_RECORD_TASK      = 6,  
    APP_LINEIN_TASK      = 7,  
    APP_RTC_TASK         = 8,  
    APP_PC_TASK          = 9,  
    APP_SPDIF_TASK       = 10,  
    APP_IDLE_TASK        = 11,  
    APP_TASK_MAX_INDEX,  
};
```

3.3 模式流程的详细介绍（以上电进入 powerno 模式为例）

1.APP 模式设置

上电设置 app 模式值

```
/* endless_loop_debug_int(); */  
ui_update_status(STATUS_POWERON);  
  
app_curr_task = APP_POWERON_TASK;  
}  
  
app_task_loop();
```

AL master apps/soundbox/app_main.c

2.Poweron 模式进入

```
void app_task_loop()
{
    while (1) {
        switch (app curr task) {
            case APP_POWERON_TASK:
                log_info("APP_POWERON_TASK \n");
                app_poweron_task();
                break;
            case APP_POWEROFF_TASK:
                log_info("APP_POWEROFF_TASK \n");
                app_poweroff_task();
                break;
            case APP_BT_TASK:
                log_info("APP_BT_TASK \n");
                app_bt_task();
                break;
            case APP_MUSIC_TASK:
                log_info("APP_MUSIC_TASK \n");
```

3.APP 模式切换

(1) 设置下一模式值

```
static int power_on_init(void)
{
    ///有些需要在开机提示完成之后再初始化的东西， 可以在这里初始化
    #if (TCFG_SPI_LCD_ENABLE)
        lcd_ui_power_on();//由ui决定切换的模式
        return 0;
    #endif

    printf(">%s, %d\n", FUNCTION, __LINE__);
    #if TCFG_APP_BT_EN
        app_task_switch_to(APP_BT_TASK);
    #else
        app_task_switch_to(APP_MUSIC_TASK);
    #endif

    return 0;
}
```

(2) 推出 APP_MSG_SWITCH_TASK 切换消息

```

0 int app_task_switch_to(u8 app_task)
1 {
2     if (app_curr_task == app_task) {
3         return false;
4     }
5
6     if (!app_task_switch_check(app_task)) {
7         return false;
8     }
9
10    if (!app_task_switch_exit_check(app_curr_task)) {
11        return false;
12    }
13
14    printf("cur --- %x \n", app_curr_task);
15    printf("new +++ %x \n", app_task);
16
17    /* if(app_next_task) */
18    /* printf("app_task_switch_to busy \n"); */
19    #if (defined SMART_BOX_EN) && (SMART_BOX_EN)
20    extern void function_change_inform(u8 app_mode, u8 ret);
21    function_change_inform(app_task, TRUE);
22    #endif
23
24    app_prev_task = app_curr_task;
25    app_next_task = app_task;
26    app_task_put_usr_msg(APP_MSG_SWITCH_TASK, 0);
27
28    return TRUE;
29 }
30
31 /**

```

4.poweron 模式退出，跳出 while（1）循环

```

while (1) {
    app_task_get_msg(msg, ARRAY_SIZE(msg), 1);
    switch (msg[0]) {
    case APP_MSG_SYS_EVENT:
        if (poweron_sys_event_handler((struct sys_event *) (msg + 1)) == false) {
            app_default_event_deal((struct sys_event *) (&msg[1])); //由common统一处理
        }
        break;
    default:
        break;
    }

    if (app_task_exitting()) {
        power_on_unint();
        return;
    }
}

```

master apps/soundbox/task_manager/power_on/power_on.c

3.4 APP 模式管理函数接口介绍

APP 模式管理函数集中在 task_manager/app_task_switch.c

1.APP 模式配置表，这里可以配置切换模式的顺序，方案根据需求定义

```
///模式配置表，这里可以配置切换模式的顺序，方案根据需求定义
static const u8 app_task_list[] = {
#ifdef TCFG_APP_BT_EN
    APP_BT_TASK,
#endif
#ifdef TCFG_APP_MUSIC_EN
    APP_MUSIC_TASK,
#endif
#ifdef TCFG_APP_FM_EN
    APP_FM_TASK,
#endif
#ifdef TCFG_APP_RECORD_EN
    APP_RECORD_TASK,
#endif
#ifdef TCFG_APP_LINEIN_EN
    APP_LINEIN_TASK,
#endif
#ifdef TCFG_APP_RTC_EN
    APP_RTC_TASK,
#endif
#ifdef TCFG_APP_PC_EN
    APP_PC_TASK,
#endif
#ifdef TCFG_APP_SPDIF_EN
    APP_SPDIF_TASK,
#endif
};
```

2.相关函数介绍

```
/**-----*/
/**@brief    模式退出检查
  @param    curr_task:当前模式
  @return    TRUE可以退出， FALSE不可以退出
  @note
*/
/**-----*/
static int app_task_switch_exit_check(u8 curr_task)
{
    int ret = false;
```

```
3  /**-----*/
4  /**@brief    模式进入检查
5   @param    app_task:目标模式
6   @return    TRUE可以进入, FALSE不可以进入
7   @note      例如一些需要设备在线的任务(music/record/linein等),
8              如果设备在线可以进入, 没有设备在线不进入可以在这里处理
9  */
10 /**-----*/
11 static int app_task_switch_check(u8 app_task)
12 {
13     int ret = false;
14     switch (app_task) {
15 #if TCFG_APP_MUSIC_EN
16     case APP_MUSIC_TASK;
```

```
/**-----*/
/**@brief    切换到上一个模式
 @param
 @return
 @note
 */
/**-----*/
void app_task_switch_prev()
{
    int i = 0;
    int cur_index = 0;

    if (app_next_task) {
```

```
/**-----*/
/**@brief    切换到下一个模式
 @param
 @return
 @note
 */
/**-----*/
void app_task_switch_next()
{
    int i = 0;
    int cur_index = 0;

    if (app_next_task) {
        printf("app_task_switch_next busy \n");
    }
```

```
/**-----*/
/**@brief  切换到指定模式
  @param  app_task:指定模式
  @return
  @note
*/
/**-----*/
int app_task_switch_to(u8 app_task)
{
    if (app_curr_task == app_task) {
        return false;
    }

    if (!app_task_switch_check(app_task)) {
        return false;
    }
}
```

```
0 /**-----*/
1 /**@brief  跳回到原来的模式
2   @param
3   @return
4   @note
5 */
6 /**-----*/
7
8 int app_task_switch_back()
9 {
10     if (app_prev_task == 0) {
11         return -EINVAL;
12     }
13     return app_task_switch_to(app_prev_task);
14 }
```

```
/**-----*/
/**@brief  模式切换退出检测
  @param
  @return  1:响应退出模式, 0:不响应
  @note
*/
/**-----*/
u8 app_task_exitting()//
{
    struct sys_event clear_key_event = {.type = SYS_KEY_EVENT, .arg = (void *)DEVICE_EVENT_FROM_KEY};
    if (app_next_task != 0) {
        app_curr_task = app_next_task;
        app_next_task = 0;
    }
}
```

```
/**-----*/
/**@brief  获取当前模式
  @param
  @return  当前模式id
  @note
*/
/**-----*/
u8 app_get_curr_task()
{
    return app_curr_task;
}
```

```
/**-----*/
/**@brief    通过指定id检查是否是当前模式
  @param
  @return    true:是当前模式, false:不是当前模式
  @note
*/
/**-----*/
u8 app_check_curr_task(u8 app)
{
    if (app == app_curr_task) {
        return true;
    }
    return false;
}
```

3.5 增加 APP 模式例子（以 music 为例）

1. 在 app_task.h 中增加模式 id（以 music 为例）

```
9 enum {
10     APP_POWERON_TASK = 1,
11     APP_POWEROFF_TASK = 2,
12     APP_BT_TASK = 3,
13     APP_MUSIC_TASK = 4,
14     APP_FM_TASK = 5,
15     APP_RECORD_TASK = 6,
16     APP_LINEIN_TASK = 7,
17     APP_RTC_TASK = 8,
18     APP_PC_TASK = 9,
19     APP_SPDIF_TASK = 10,
20     APP_IDLE_TASK = 11,
21     APP_TASK_MAX_INDEX,
22 };
```

2. 将新加的模式 id 加入到 app_task_switch.c 中模式配置表 app_task_list

```
6 ///模式配置表，这里可以配置切换模式的顺序，方案根据需求定义
7 static const u8 app_task_list[] = {
8     #if TCFG_APP_BT_EN
9         APP_BT_TASK,
10    #endif
11    #if TCFG_APP_MUSIC_EN
12        APP_MUSIC_TASK,
13    #endif
14    #if TCFG_APP_FM_EN
15        APP_FM_TASK,
16    #endif
17    #if TCFG_APP_RECORD_EN
18        APP_RECORD_TASK,
19    #endif
20    #if TCFG_APP_LINEIN_EN
21        APP_LINEIN_TASK,
22    #endif
23    #if TCFG_APP_RTC_EN
24        APP_RTC_TASK,
25    #endif
26    #if TCFG_APP_PC_EN
27        APP_PC_TASK,
28    #endif
29    #if TCFG_APP_SPDIF_EN
30        APP_SPDIF_TASK,
31    #endif
32 };
```

- 3.参考 task_key.c 中参考添加模式按键转换表（ad、io、ir 等）
- 4.在 task_manager 中添加对应的模式目录（及对应的头文件目录）
- 5.实现模式相关接口（参考已有模式，以下以 music 为例进行说明）

① 实现以下基础必要接口：

void app_music_task()

int music_app_check(void)

static int music_sys_event_handler(struct sys_event *event)

static int music_key_event_opr(struct sys_event *event)

② 模式主循环内完成以下基础操作（app_music_task）

获取消息

响应消息及事件

响应模式内部消息及事件

响应公共消息及事件

```
void app_music_task()
{
    int res;
    int msg[32];
    music_task_start(); // 初始化, 非必要, 根据具体情景定义

    int err = tone_play_with_callback_by_name(tone_table[IDEX_TONE_MUSIC], 1, music_tone_play_end_callback, (void *)IDEX_TONE_MUSIC);
    if (err) {
        music_player_play_start(); // 播放模式提示音, 非必要, 如果要的话参考music.c的案例实现
    }

    while (1) { // 模式主循环
        app_task_get_msg(msg, ARRAY_SIZE(msg), 1); // 获取消息
        switch (msg[0]) {
            case APP_MSG_SYS_EVENT: // 处理系统case消息
                if (music_sys_event_handler((struct sys_event *)&msg[1])) == false { // 模式内部消息拦截, 如果不拦截, 给公共消息处理响应
                    app_default_event_deal((struct sys_event *)&msg[1]); // 必要, 响应公共消息处理
                }
                break;
            default:
                break;
        }
        if (app_task_exitting()) { // 必要, 模式退出检测处理
            music_task_close(); // 模式退出内部释放处理, 非必要, 如果有初始化, 就一定要注意释放操作
            return;
        }
    }
}
```

- ③ 在 app_main.c 中调用对应的模式主循环接口 (app_music_task)

```
38 void app_task_loop()
39 {
40     while (1) {
41         switch (app_curr_task) {
42             case APP_POWERON_TASK:
43                 log_info("APP_POWERON_TASK \n");
44                 app_poweron_task();
45                 break;
46             case APP_POWEROFF_TASK:
47                 log_info("APP_POWEROFF_TASK \n");
48                 app_poweroff_task();
49                 break;
50             case APP_BT_TASK:
51                 log_info("APP_BT_TASK \n");
52                 app_bt_task();
53                 break;
54             case APP_MUSIC_TASK:
55                 log_info("APP_MUSIC_TASK \n");
56                 app_music_task();
57                 break;
58             case APP_FM_TASK:
59                 log_info("APP_FM_TASK \n");
60                 app_fm_task();
61                 break;
62             case APP_RECORD_TASK:
63                 log_info("APP_RECORD_TASK \n");
64                 app_record_task();
65                 break;
66         }
67     }
68 }
```

NORMAL ▶ master ▶ apps/soundbox/app_main.c

- ④ app_check 接口的实现 (music 为例)

app_check 其实是在切换模式的时候, 是否满足条件进入该模式, music 模式进入条件是判断是否有可以播放的设备在线, 故接口实现如下:

```
634 int music_app_check(void)
635 {
636     if (dev_manager_get_total(1)) {
637         return true;
638     }
639     return false;
640 }
```

- ⑤ 在 app_task_switch_check 调用 app_check (music 为例)

```
107 static int app_task_switch_check(u8 app_task)
108 {
109     int ret = false;
110     switch (app_task) {
111 #if TCFG_APP_MUSIC_EN
112         case APP_MUSIC_TASK:
113             ret = music_app_check();
114             break;
115 #endif
116 #if TCFG_APP_LINEIN_EN
117         case APP_LINEIN_TASK:
118             ret = linein_app_check();
119             break;
120 #endif
121 #if TCFG_APP_PC_EN
```

Chapter 4 APP 消息管理

4.1 系统事件管理

1、系统事件的分类

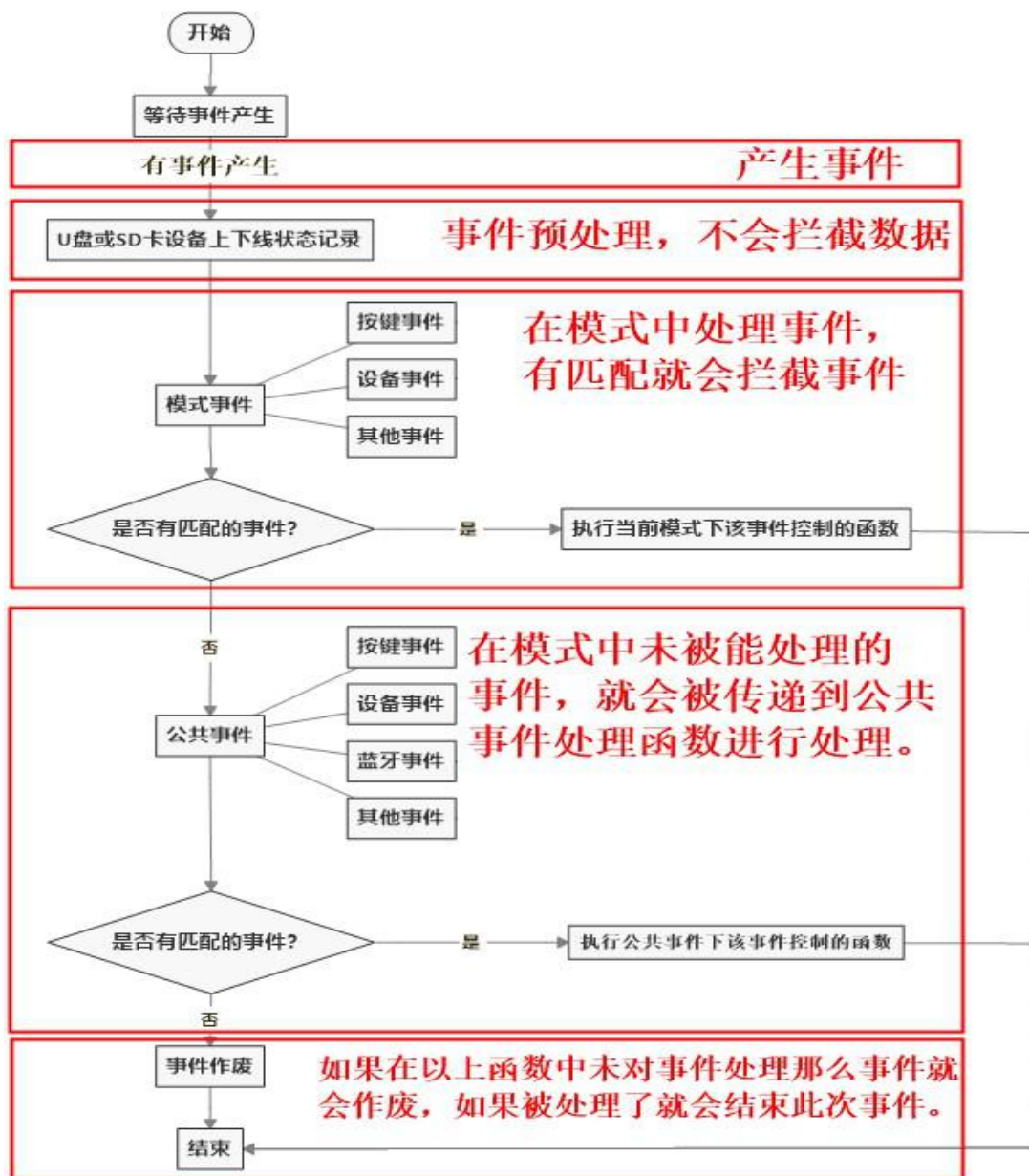
系统事件分为系统按键事件、系统设备事件以及系统蓝牙事件，为了更好地管理，还把这三大事件分为模式里的系统事件类和每个模式都有的公共系统事件类。流程图如下：

特别说明：

①系统产生事件后，会先到模式里的系统事件查询是否有匹配的事件，如有，则执行相应的函数并且不再执行公共系统事件类，然后结束事件，如无，则到公共系统事件查询是否有匹配的事件，如有，则执行相应的函数，然后结束事件，如无，则视为事件作废，结束事件。

②蓝牙模式的设备事件如果不做处理，那么会把事件丢到公共事件中的公共蓝牙事件去处理，而不是丢到公共事件的公共设备事件去处理。

以下为系统事件的管理机制：



4.2 提供的 APP 消息处理函数说明

1.提供的消息函数接口

```
5
6
7 //app自定义消息发送接口
8 int app_task_put_usr_msg(int msg, int arg_num, ...);
9
10 //app消息获取接口(block参数为0表示内部pend, 1直接返回)
11 void app_task_get_msg(int *msg, int msg_size, int block);
12
13 //app按键消息发送接口
14 int app_task_put_key_msg(int msg, int value);
15
16 //app清理按键消息接口
17 void app_task_clear_key_msg();
18
```

2.消息处理机制

(1) 获取消息

```
void app_poweron_task()
{
    int msg[32];

    UI_SHOW_MENU(MENU_POWER_UP, 0, 0, NULL);

    int err = tone_play_with_callback_by_name(tone_table[IDEX_TONE_POWER_ON], 1, tone_play_end_callback, (void *)IDEX_TONE_POWER_ON);
    if (err) { //提示音没有,播放失败,直接init流程
        power_on_init();
    }

    while (1) {
        app_task_get_msg(msg, ARRAY_SIZE(msg), 1);
        switch (msg[0]) {
            case APP_MSG_SYS_EVENT:
                if (poweron_sys_event_handler((struct sys_event *)(&msg[1])) == false) {
                    app_default_event_deal((struct sys_event *)(&msg[1])); //由common统一处理
                }
                break;
            default:
                break;
        }

        if (app_task_exitting()) {
            power_on_unint();
            return;
        }
    }
}
```

(2) 推出 app 按键处理消息（非物理按键触发）

```
break;
}
if (logo && (0 == memcmp(logo, evt_logo, str_len))) {
    ///相同的设备才响应
    if (music_player_get_playing_breakpoint(breakpoint, 1) == true) {
        breakpoint_vm_write(breakpoint, logo);
    }
    ///停止解码,防止设备掉线后还继续使用
    music_player_stop(1);
    ///重新选择活动设备播放
    app_task_put_key_msg(KEY_MUSIC_PLAYER_START, 0);///卸载了设备再执行
    log_i("KEY_MUSIC_PLAYER_START_AFTER_UMOUNT\n");
}
} else {
    ///新设备上线,先记录当前设备断点,然后播放活动设备
    if (music_player_get_playing_breakpoint(breakpoint, 1) == true) {
        breakpoint_vm_write(breakpoint, logo);
    }
    ///停止解码,播放新活动设备
    music_player_stop(1);
    app_task_put_key_msg(KEY_MUSIC_PLAYER_START, 0);
    log_i("KEY_MUSIC_PLAYER_START_AFTER_MOUNT\n");
    ///先挂载了设备再执行
}
break;
default://switch((u32)event->arg)
break;
```

3. app 自定义消息发送接口

(1) 消息值定义

```
4
5 enum {
6     APP_MSG_SYS_EVENT = Q_EVENT,
7
8     /* 用户自定义消息 */
9     APP_MSG_SWITCH_TASK = Q_USER + 1,
10    APP_MSG_USER          = Q_USER + 2,
11
12    // 添加消息值
13    // eg:
14    // APP_MSG_TEST      = Q_USER + 3,
15    //
16
17 },
18
19
20 //切换到前一个有效模式
21 void app_task_switch_prev();
22 //切换到下一个有效模式
```

(2) 消息处理

以蓝牙模式为例（如果消息其他模式都需要处理，就要在需要处理的模式都添加）

```
void app_bt_task()
{
    int res;
    int msg[32];
    ui_update_status(STATUS_EXIT_LOWPOWER);

    if (!__this->cmd_flag) { //蓝牙后台拉回蓝牙模式不播放提示音
        tone_play_by_path(tone_table[INDEX_TONE_BT_MODE], 1);
    }

    bt_task_start();

    while (1) {
        app_task_get_msg(msg, ARRAY_SIZE(msg), 1);

        switch (msg[0]) {
            case APP_MSG_SYS_EVENT:
                if (bt_sys_event_handler((struct sys_event *) (msg + 1)) == false) {
                    app_default_event_deal((struct sys_event *) (&msg[1]));
                }
                break;

            /* case APP_MSG_TEST:
            /*      // deal */
            /*      break; */

            default:
                break;
        }

        if (app_task_exitting()) {
            bt_task_close();
            __this->wait_exit = 1;
        }

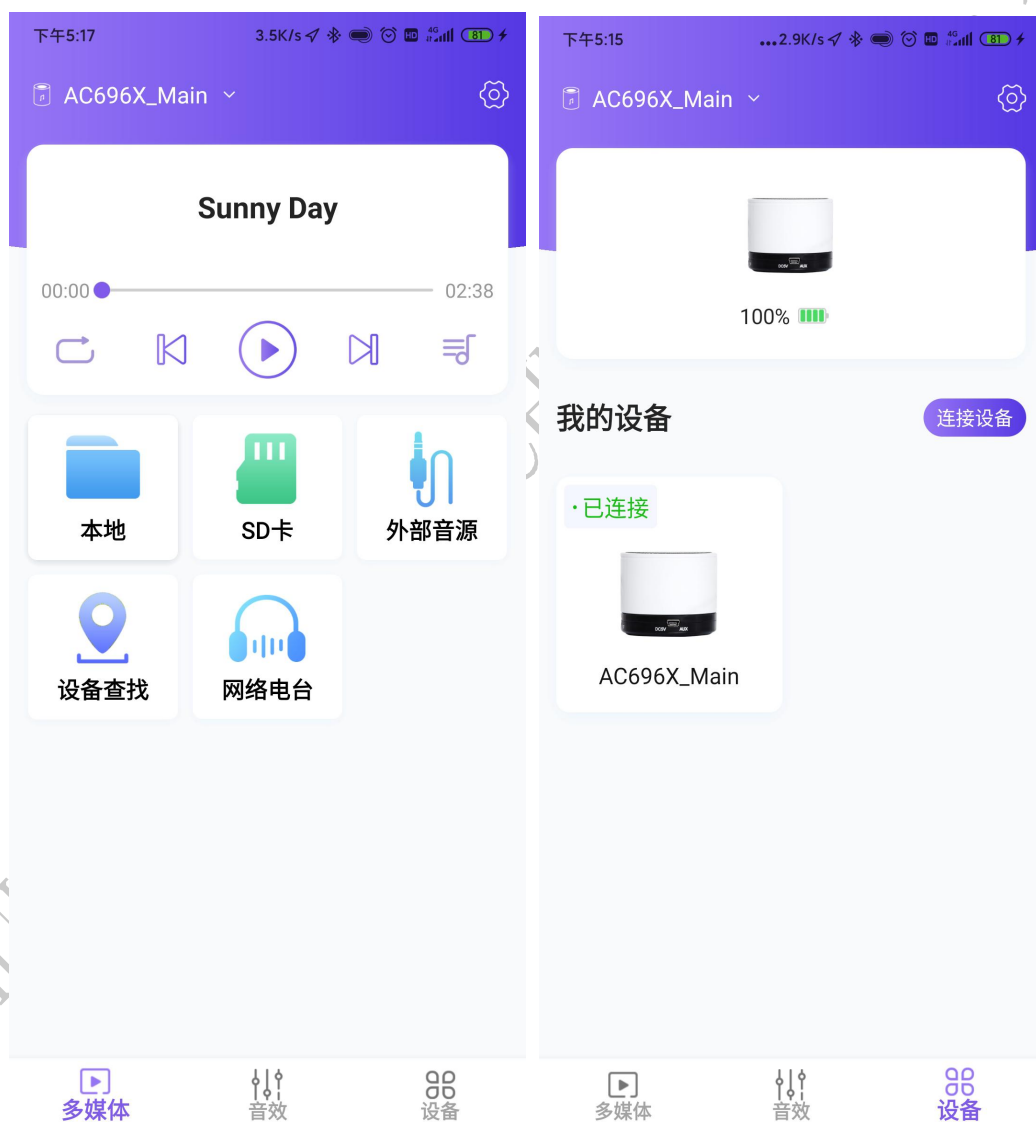
        if (__this->wait_exit) { //开始退出
            if (!__this->exiting) { ///等待蓝牙退出处理完成
                return;
            }
        }
    }
}
```

Chapter 5 杰理之家 APP

5.1 编写目的

该章节主要描述 AC696N 系列芯片杰理之家 APP 介绍，为用户进行二次开发提供参考

5.2 APP 介绍



5.3 文件目录说明

```
▼ soundbox/
  ▶ aec/
  ▶ app_api/
  ▶ board/
  ▶ bt/
  ▶ charge_box/
  ▶ common/
  ▶ fm/
  ▶ font/
  ▶ include/
  ▶ init/
  ▶ linein/
  ▶ log_config/
  ▶ music/
  ▶ pc/
  ▶ power_manage/
  ▶ record/
  ▶ rtc/
  ▼ smartbox/
    ▶ browser/
    ▶ bt_manage/
    ▶ cmd_data_deal/
    ▶ func_cmd/
    ▶ smartbox_setting/
    ▶ smartbox_setting_opt/
    ▶ smartbox_update/
    config.c
    event.c
    feature.c
    function.c
    Makefile
    smartbox.c
    smartbox_rcsp_manage.c
    smartbox_rcsp_manage.h
    switch_device.c
  ▶ soundcard/
  ▶ spdif/
```

协议代码: soundbox/smartbox

目录说明:

-browser

目录浏览相关目录, 存放目录浏览相关代码。

- bt_manage

透传接口目录, 存放透传相关代码。

- cmd_data_deal

rcsp 协议目录, 存放 rcsp 交互相关代码。

- func_cmd

各种模式 (如蓝牙模式、音乐模式) 功能目录。

- smartbox_setting

功能设置目录, 存放各种功能具体实现的代码。

- smartbox_setting_opt

功能设置操作目录, 存放公共功能设置相关的代码。

- smartbox_update

OTA 升级目录, 存放升级相关的代码。

-其他

其他文件是 SMARTBOX 相关的公共操作。

5.4 功能开关、配置

在 `soundbox/board/br25/board_ac696x_smartbox/board_ac696x_smartbox.h` 里

`#define CONFIG_APP_BT_ENABLE` （功能总开关）

`#define SMART_BOX_EN 1`

```
*****//
//                               AI配置                               //
*****//
#define CONFIG_APP_BT_ENABLE // AI功能、流程总开关

#ifndef CONFIG_APP_BT_ENABLE
#define TRANS_DATA_EN 0
#define SMART_BOX_EN 1
#else
#define TRANS_DATA_EN 0
#define SMART_BOX_EN 0
#endif

#if (SMART_BOX_EN) //rcsp需要打开ble
#define CONFIG_DOUBLE_BANK_ENABLE 0
#define RCSP_UPDATE_EN 1 //是否支持rcsp升级
#define OTA_TWS_SAME_TIME_ENABLE 0 //是否支持TWS同步升级
#define UPDATE_MDS_ENABLE 1 //升级是否支持MDS校验
#define RCSP_FILE_OPT 1
#define JL_EARPHONE_APP_EN 1
#define TCFG_LFN_EN 1
#define TCFG_BS_DEV_PATH_EN 1
#else
#define OTA_TWS_SAME_TIME_ENABLE 0
#define RCSP_UPDATE_EN 0
#define UPDATE_MDS_ENABLE 0 //升级是否支持MDS校验
#define RCSP_FILE_OPT 0
#define JL_EARPHONE_APP_EN 0
#endif

#if RCSP_UPDATE_EN
#define APP_UPDATE_EN 0 //需要使用APP升级的话要把该宏打开
#else
#define APP_UPDATE_EN 0 //客户如需要开发自己的app升级协议需要把这个宏打开,并提供升级需要的read\seek等接口,具体请参照说明文档
#endif
```

在 `soundbox/board/br25/board_ac696x_smartbox/board_ac696x_smartbox.h` 里

关于具体支持模式使能相关的宏设置

```
*****//
//                               app 配置                               //
*****//
#define TCFG_APP_BT_EN 1
#define TCFG_APP_MUSIC_EN 1
#define TCFG_APP_LINEIN_EN 1
#define TCFG_APP_FM_EN 0
#define TCFG_APP_PC_EN 0
#define TCFG_APP_RTC_EN 0
#define TCFG_APP_RECORD_EN 0
#define TCFG_APP_SPDIF_EN 0
#define TCFG_APP_RTC_SIMULATE_EN 0
```

关于 RCSP 功能具体模块设置使能相关的宏设置

在 `soundbox/smartbox/bt_manage/bt_trans_data/le_smartbox_module.h` 里

```
//rcsp功能模块使能
#define RCSP_ADV_NAME_SET_ENABLE          1
#define RCSP_ADV_KEY_SET_ENABLE           0
#define RCSP_ADV_LED_SET_ENABLE           1
#define RCSP_ADV_MIC_SET_ENABLE           0
#define RCSP_ADV_WORK_SET_ENABLE          0
#define RCSP_ADV_EQ_SET_ENABLE            1
#define RCSP_ADV_MUSIC_INFO_ENABLE        1
#define RCSP_ADV_HIGH_LOW_SET             1
#define RCSP_ADV_PRODUCT_MSG_ENABLE       1
#define RCSP_ADV_FIND_DEVICE_ENABLE       1
```

5.5 命令、数据 API 介绍

5.5.1 已有命令

```
/*
 注意：不能在这个枚举里加代码
 注意：不能在这个枚举里加代码
 注意：不能在这个枚举里加代码
*/
enum {
    JL_OPCODE_DATA = 0x01,
    JL_OPCODE_GET_TARGET_FEATURE_MAP = 0x02,
    JL_OPCODE_GET_TARGET_FEATURE = 0x03,
    // JL_OPCODE_SWITCH_DEVICE = 0x04,
    JL_OPCODE_DISCONNECT_EDR = 0x06,
    JL_OPCODE_SYS_INFO_GET = 0x07,
    JL_OPCODE_SYS_INFO_SET = 0x08,
    JL_OPCODE_SYS_INFO_AUTO_UPDATE = 0x09,
    JL_OPCODE_CALL_REQUEST = 0xa,
    JL_OPCODE_SWITCH_DEVICE = 0x0b,
    JL_OPCODE_FILE_BROWSE_REQUEST_START = 0x0c,
    JL_OPCODE_FILE_BROWSE_REQUEST_STOP = 0x0d,
    JL_OPCODE_FUNCTION_CMD = 0x0e,

    JL_OPCODE_SYS_OPEN_BT_SCAN = 0x12,
    JL_OPCODE_SYS_UPDATE_BT_STATUS = 0x13,
    JL_OPCODE_SYS_STOP_BT_SCAN = 0x14,
    JL_OPCODE_SYS_BT_CONNECT_SPEC = 0x15,
    JL_OPCODE_SYS_EMITTER_BT_CONNECT_STATUS = 0x20, // 原来0x19
    JL_OPCODE_SYS_FIND_DEVICE = 0x19,

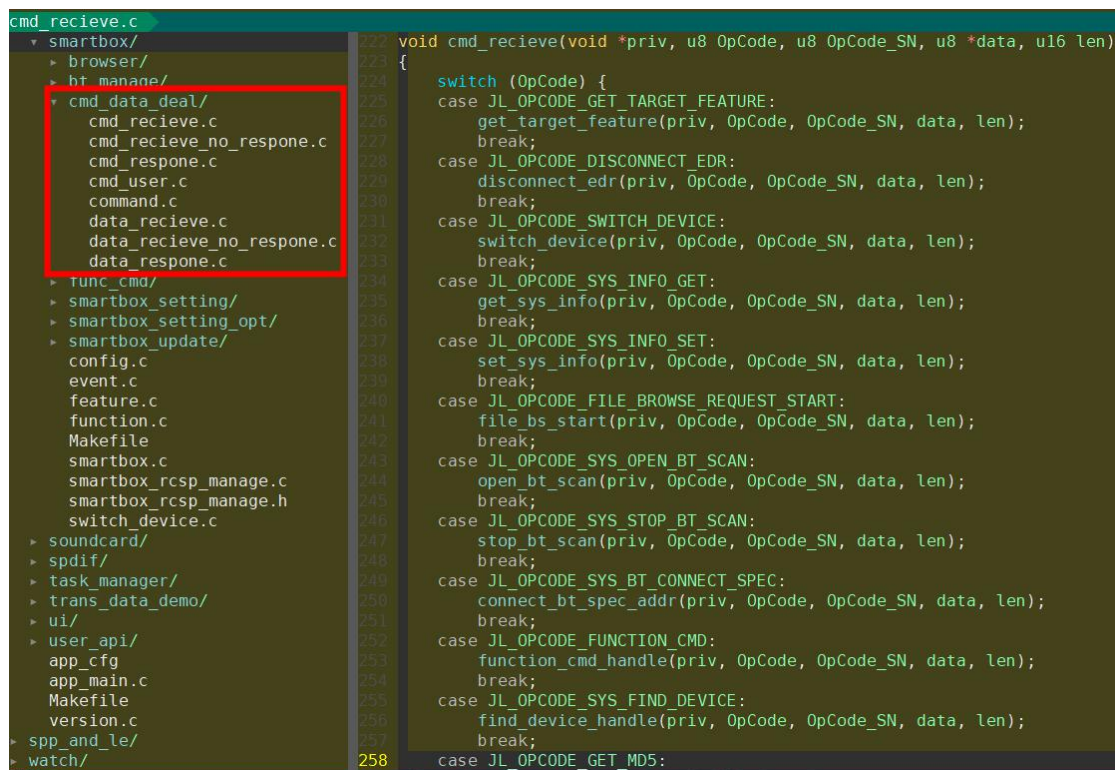
    JL_OPCODE_GET_MD5 = 0xd4,
    JL_OPCODE_LOW_LATENCY_PARAM = 0xd5,

    JL_OPCODE_CUSTOMER_USER = 0xff,
};
/*
 注意：不能在这个枚举里加代码
 注意：不能在这个枚举里加代码
 注意：不能在这个枚举里加代码
*/
```

其中 JL_OPCODE_CUSTOMER_USER = 0xFF 用于自定义命令扩展

注意： 不能用其他非 0xFF 的命令号来自定义命令

5.5.2 小机和手机交互的处理



```

cmd_recieve.c
smartbox/
  browser/
  bt_manage/
  cmd_data_deal/
    cmd_recieve.c
    cmd_recieve_no_response.c
    cmd_response.c
    cmd_user.c
    command.c
    data_recieve.c
    data_recieve_no_response.c
    data_response.c
  func_cmd/
  smartbox_setting/
  smartbox_setting_opt/
  smartbox_update/
    config.c
    event.c
    feature.c
    function.c
    Makefile
    smartbox.c
    smartbox_rcsp_manage.c
    smartbox_rcsp_manage.h
    switch_device.c
  soundcard/
  spdif/
  task_manager/
  trans_data_demo/
  ui/
  user_api/
    app_cfg
    app_main.c
    Makefile
    version.c
  spp_and_le/
  watch/

222 void cmd_recieve(void *priv, u8 OpCode, u8 OpCode_SN, u8 *data, u16 len)
223 {
224     switch (OpCode) {
225     case JL_OPCODE_GET_TARGET_FEATURE:
226         get_target_feature(priv, OpCode, OpCode_SN, data, len);
227         break;
228     case JL_OPCODE_DISCONNECT_EDR:
229         disconnect_edr(priv, OpCode, OpCode_SN, data, len);
230         break;
231     case JL_OPCODE_SWITCH_DEVICE:
232         switch_device(priv, OpCode, OpCode_SN, data, len);
233         break;
234     case JL_OPCODE_SYS_INFO_GET:
235         get_sys_info(priv, OpCode, OpCode_SN, data, len);
236         break;
237     case JL_OPCODE_SYS_INFO_SET:
238         set_sys_info(priv, OpCode, OpCode_SN, data, len);
239         break;
240     case JL_OPCODE_FILE_BROWSE_REQUEST_START:
241         file_bs_start(priv, OpCode, OpCode_SN, data, len);
242         break;
243     case JL_OPCODE_SYS_OPEN_BT_SCAN:
244         open_bt_scan(priv, OpCode, OpCode_SN, data, len);
245         break;
246     case JL_OPCODE_SYS_STOP_BT_SCAN:
247         stop_bt_scan(priv, OpCode, OpCode_SN, data, len);
248         break;
249     case JL_OPCODE_SYS_BT_CONNECT_SPEC:
250         connect_bt_spec_addr(priv, OpCode, OpCode_SN, data, len);
251         break;
252     case JL_OPCODE_FUNCTION_CMD:
253         function_cmd_handle(priv, OpCode, OpCode_SN, data, len);
254         break;
255     case JL_OPCODE_SYS_FIND_DEVICE:
256         find_device_handle(priv, OpCode, OpCode_SN, data, len);
257         break;
258     case JL_OPCODE_GET_MD5:

```

小机接收到的命令，统一在 cmd_recieve.c 的 cmd_recieve 函数中（对已有命令）进行处理。

其中较为常用的有 JL_OPCODE_GET_TARGET_FEATURE 、 JL_OPCODE_SYS_INFO_GET 、 JL_OPCODE_SYS_INFO_SET 和 JL_OPCODE_FUNCTION_CMD 这几个命令。

JL_OPCODE_GET_TARGET_FEATURE 命令是用于设置 SMARTBOX 的特性。对应会调用到 soundbox/smartbox/feature.c 的 target_feature_mask_get_tab 列表中所有的函数。

```

15
16 static const attr_get_func target_feature_mask_get_tab[FEATURE_ATTR_TYPE_MAX] = {
17     [FEATURE_ATTR_TYPE_PROTOCOL_VERSION] = target_feature_attr_protocol_version,
18     [FEATURE_ATTR_TYPE_SYS_INFO] = target_feature_attr_sys_info,
19     [FEATURE_ATTR_TYPE_EDR_ADDR] = target_feature_attr_edr_addr,
20     [FEATURE_ATTR_TYPE_PLATFORM] = target_feature_attr_platform,
21     [FEATURE_ATTR_TYPE_FUNCTION_INFO] = target_feature_function_info,
22     [FEATURE_ATTR_TYPE_DEV_VERSION] = target_feature_dev_version,
23     [FEATURE_ATTR_TYPE_SDK_TYPE] = target_feature_sdk_type,
24     [FEATURE_ATTR_TYPE_UBOOT_VERSION] = target_feature_uboot_version,
25     [FEATURE_ATTR_TYPE_DOUBLE_PARTITION] = target_feature_ota_double_partition,
26     [FEATURE_ATTR_TYPE_UPDATE_STATUS] = target_feature_ota_update_status,
27     [FEATURE_ATTR_TYPE_DEV_VID_PID] = target_feature_pid_vid,
28     [FEATURE_ATTR_TYPE_DEV_AUTHKEY] = target_feature_authkey,
29     [FEATURE_ATTR_TYPE_DEV_PROCODE] = target_feature_procode,
30     [FEATURE_ATTR_TYPE_DEV_MAX_MTU] = target_feature_mtu,
31     [FEATURE_ATTR_TYPE_CONNECT_BLE_ONLY] = target_feature_ble_only,
32     [FEATURE_ATTR_TYPE_BT_EMITTER_INFO] = target_feature_bt_emitter_info,
33     [FEATURE_ATTR_TYPE_MD5_GAME_SUPPORT] = target_feature_md5_game_support,
34 };
35
36 u32 target_feature_parse_packet(void *priv, u8 *buf, u16 buf_size, u32 mask)
37 {
38     printf("target_feature_parse_packet, mask = %x\n", mask);
39     return attr_get(priv, buf, buf_size, target_feature_mask_get_tab, FEATURE_ATTR_TYPE_MAX, mask);
40 }
41 #endif//SMART_BOX_EN
42
43
NORMAL master ~/BR22_SDK/SDK/apps/soundbox/smartbox/feature.c

```

JL_OPCODE_SYS_INFO_GET 命令是用于获取 SMARTBOX 系统信息的命令。对应会调用到 soundbox/smartbox/function.c 的 target_common_function_get_tab 列表中的所有函数。

```

70
71 static const attr_get_func target_common_function_get_tab[COMMON_FUNCTION_ATTR_TYPE_MAX] = {
72     [COMMON_FUNCTION_ATTR_TYPE_BATTERY] = common_function_attr_battery_get,
73     [COMMON_FUNCTION_ATTR_TYPE_VOL] = common_function_attr_vol_get,
74     [COMMON_FUNCTION_ATTR_TYPE_DEV_INFO] = common_function_attr_dev_info_get,
75     [COMMON_FUNCTION_ATTR_TYPE_ERROR_STATS] = NULL,
76     [COMMON_FUNCTION_ATTR_TYPE_EQ_INFO] = common_function_attr_eq_param_get,
77     [COMMON_FUNCTION_ATTR_TYPE_BS_FILE_TYPE] = common_function_attr_bs_file_type,
78     [COMMON_FUNCTION_ATTR_TYPE_FUNCTION_MODE] = common_function_attr_function_mode_get,
79     [COMMON_FUNCTION_ATTR_TYPE_FMTX_FREQ] = common_function_attr_fmtx_freq_get,
80     [COMMON_FUNCTION_ATTR_TYPE_BT_EMITTER_SW] = common_function_attr_bt_emitter_sw_get,
81     [COMMON_FUNCTION_ATTR_TYPE_BT_EMITTER_CONNECT_STATES] = common_function_attr_bt_emitter_connect_state_get,
82     [COMMON_FUNCTION_ATTR_TYPE_HIGH_LOW_SET] = common_function_attr_high_low_get,
83     [COMMON_FUNCTION_ATTR_TYPE_PRE_FETCH_ALL_EQ_INFO] = common_function_attr_pre_fetch_all_eq_info_get,
84     [COMMON_FUNCTION_ATTR_TYPE_PHONE_SCO_STATE_INFO] = common_function_attr_phone_sco_state_info_get,
85 };
86
87
88
89 static bool smartbox_common_function_set(void *priv, u8 *data, u16 len)
90 {
91     printf("smartbox_common_function_set\n");
92     struct smartbox *smart = (struct smartbox *)priv;
93     if (smart == NULL) {
94         return false;
95     }
96     put_buf(data, len);
97     attr_set(priv, data, len, common_function_set_tab, COMMON_FUNCTION_ATTR_TYPE_MAX);
98 }
99
NORMAL master ~/BR22_SDK/SDK/apps/soundbox/smartbox/function.c

```

JL_OPCODE_SYS_INFO_SET 命令是用与设置 SMARTBOX 系统信息的命令。对应会调用到 soundbox/smartbox/function.c 的 common_function_set_tab 列表中的所有函数。

```

1 static const attr_set_func common_function_set_tab[COMMON_FUNCTION_ATTR_TYPE_MAX] = {
2     [COMMON_FUNCTION_ATTR_TYPE_BATTERY] = NULL,
3     [COMMON_FUNCTION_ATTR_TYPE_VOL] = common_function_attr_vol_set,
4     [COMMON_FUNCTION_ATTR_TYPE_DEV_INFO] = NULL,
5     [COMMON_FUNCTION_ATTR_TYPE_ERROR_STATS] = NULL,
6     [COMMON_FUNCTION_ATTR_TYPE_EQ_INFO] = common_function_attr_eq_set, //NULL,
7     [COMMON_FUNCTION_ATTR_TYPE_BS_FILE_TYPE] = NULL,
8     [COMMON_FUNCTION_ATTR_TYPE_FUNCTION_MODE] = NULL,
9     [COMMON_FUNCTION_ATTR_TYPE_FMTX_FREQ] = common_function_attr_fmtx_freq_set,
10    [COMMON_FUNCTION_ATTR_TYPE_BT_EMITTER_SW] = common_function_attr_bt_emitter_sw_set,
11    [COMMON_FUNCTION_ATTR_TYPE_BT_EMITTER_CONNECT_STATES] = common_function_attr_bt_emitter_connect_state_set,
12    [COMMON_FUNCTION_ATTR_TYPE_HIGH_LOW_SET] = common_function_attr_high_low_set,
13    [COMMON_FUNCTION_ATTR_TYPE_PRE_FETCH_ALL_EQ_INFO] = NULL,
14    [COMMON_FUNCTION_ATTR_TYPE_PHONE_SCO_STATE_INFO] = NULL,
15 };
16
17
18
19 static u32 common_function_attr_battery_get(void *priv, u8 attr, u8 *buf, u16 buf_size, u32 offset)
20 {
21     u32 rlen = 0;
22     extern u8 get_vbat_percent(void);
23     u8 vbat = get_vbat_percent();
24     rlen = add_one_attr(buf, buf_size, offset, attr, &vbat, sizeof(vbat));
25     return rlen;
26 }
27
28 static u32 common_function_attr_vol_get(void *priv, u8 attr, u8 *buf, u16 buf_size, u32 offset)
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

JL_OPCODE_FUNCTION_CMD 命令是用于切换模式和每个模式具体的设置行为交互的指令。

该命令分别调用到 `soundbox/smartbox/function` 中的 `smartbox_function_cmd_set` 函数进行模式切换和 `set_tab` 列表进行每个模式具体的设置。而每个模式具体的设置行为对应的代码就是存放在 `func_cmd` 目录中。

```

1 function.c
2 smartbox/
3   browser/
4   bt_manage/
5   cmd_data_deal/
6   func_cmd/
7     bt_func.c
8     fm_func.c
9     linein_func.c
10    Makefile
11    music_func.c
12    rtc_func.c
13   smartbox_setting/
14   smartbox_setting_opt/
15   smartbox_update/
16   config.c
17   event.c
18   feature.c
19   function.c
20   Makefile
21   smartbox.c
22   smartbox_rcsp_manage.c
23   smartbox_rcsp_manage.h
24   switch_device.c
25   soundcard/
26   spdif/
27   task_manager/
28   trans_data_demo/
29   ui/
30   user_api/
31   app_cfg
32   app_main.c
33   Makefile
34   version.c
35   spp_and_le/
36   watch/
37   app_cfg
38   Makefile
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

在 `cmd_recieve` 函数中，还会调用到关于弹出和升级相关的处理。弹窗对应的处理是

`soundbox/smartbox/smartbox_setting_opt/smartbox_adv_bluetooth.c` 文件夹，升级对应的处理是对应 `smartbox_update` 文件目录。

```

1 cmd_recieve.c
2 power_manage/
3 record/
4 rtc/
5 smartbox/
6 browser/
7 bt_manage/
8 cmd_data_deal/
9 cmd_recieve.c
10 cmd_recieve_no_response.c
11 cmd_response.c
12 cmd_user.c
13 command.c
14 data_recieve.c
15 data_recieve_no_response.c
16 data_response.c
17 func_cmd/
18 smartbox_setting/
19 smartbox_setting_opt/
20 Makefile
21 smartbox_adv.bluetooth.c
22 smartbox_adv.bluetooth.h
23 smartbox_setting_opt.c
24 smartbox_setting_opt.h
25 smartbox_setting_sync.c
26 smartbox_setting_sync.h
27 smartbox_update/
28 Makefile
29 rcsp_ch_loader_download.c
30 smartbox_update.c
31 smartbox_update.h
32 smartbox_update_tws.c
33 smartbox_update_tws.h
34 config.c
35 event.c
36 feature.c
37 function.c
38 Makefile
39
40 case JL_OPCODE_SYS_STOP_BT_SCAN:
41     stop_bt_scan(priv, OpCode, OpCode_SN, data, len);
42     break;
43 case JL_OPCODE_SYS_BT_CONNECT_SPEC:
44     connect_bt_spec_addr(priv, OpCode, OpCode_SN, data, len);
45     break;
46 case JL_OPCODE_FUNCTION_CMD:
47     function_cmd_handle(priv, OpCode, OpCode_SN, data, len);
48     break;
49 case JL_OPCODE_SYS_FIND_DEVICE:
50     find_device_handle(priv, OpCode, OpCode_SN, data, len);
51     break;
52 case JL_OPCODE_GET_MD5:
53     get_md5_handle(priv, OpCode, OpCode_SN, data, len);
54     break;
55 case JL_OPCODE_LOW_LATENCY_PARAM:
56     get_low_latency_param(priv, OpCode, OpCode_SN, data, len);
57     break;
58 case JL_OPCODE_CUSTOMER_USER:
59     smartbox_user_cmd_recieve(priv, OpCode, OpCode_SN, data, len);
60     break;
61 default:
62     break;
63 #endif //RCSP_SMARTBOX_ADV_EN
64 if (0 == JL_smartbox_adv_cmd_resp(priv, OpCode, OpCode_SN, data, len)) {
65     break;
66 }
67 #endif
68 #if RCSP_UPDATE_EN
69 if (0 == JL_rcsp_update_cmd_resp(priv, OpCode, OpCode_SN, data, len)) {
70     break;
71 }
72 #endif
73 #endif //SMART_BOX_EN
74 break;
75 }
76 }
77 #endif
78
79 #endif //SMART_BOX_EN
80
81 apps/soundbox/smartbox/cmd_data_deal/cmd_recieve.c
  
```

如需添加自定义命令，可在 soundbox/smartbox/cmd_data_deal/cmd_user.c 文件的 smartbox_user_cmd_recieve 函数分支进行添加自定义代码（该函数会在 cmd_recieve 函数的 JL_OPCODE_CUSTOMER_USER 分支被调用）。

```

1 cmd_recieve.c
2 cmd_user.c
3 smartbox/
4 browser/
5 bt_manage/
6 cmd_data_deal/
7 cmd_recieve.c
8 cmd_recieve_no_response.c
9 cmd_response.c
10 cmd_user.c
11 command.c
12 data_recieve.c
13 data_recieve_no_response.c
14 data_response.c
15 func_cmd/
16 smartbox_setting/
17 smartbox_setting_opt/
18 smartbox_update/
19 config.c
20 event.c
21 feature.c
22 function.c
23 Makefile
24 smartbox.c
25 smartbox_rcsp_manage.c
26 smartbox_rcsp_manage.h
27 switch_device.c
28 soundcard/
29 spdif/
30 task_manager/
31 trans_data_demo/
32 ui/
33 user_api/
34 app_cfg
35 app_main.c
36 Makefile
37 version.c
38 spp_and_le/
39 watch/
40 app_cfg
41 Makefile
42 cpu/
43 include_lib/
44 jenkins/
45 lib/
46 tools/
47 AC693X_Profile添加说明文档..pd
48 co_tag.sh*
49 compile_commands.json
50 cscope.files
51 cscope.out
52 dumpfile.sh*
53
54 #include "smartbox/cmd_user.h"
55
56 /**@brief smartbox自定义命令数据接收处理
57 * @param priv:全局smartbox结构体, OpCode:当前命令, OpCode_SN:当前的SN值, data:数据, len:数据长度
58 * @return
59 * @note 二次开发需要增加自定义命令, 通过JL_OPCODE_CUSTOMER_USER进行扩展, 不要定义这个命令以外的命令, 避免后续SDK更新导致命令冲突
60 */
61 void smartbox_user_cmd_recieve(void *priv, u8 OpCode, u8 OpCode_SN, u8 *data, u16 len)
62 {
63     //自定义数据接收
64     printf("%s:", FUNCTION);
65     put_buf(data, len);
66     #if 0
67     //以下是发送测试代码
68     u8 test_send_buf[] = {0x04, 0x05, 0x06};
69     smartbox_user_cmd_send(test_send_buf, sizeof(test_send_buf));
70     #endif
71     JL_CMD_response_send(OpCode, JL_PRO_STATUS_SUCCESS, OpCode_SN, NULL, 0);
72 }
73
74 /**@brief smartbox自定义命令数据发送接口
75 * @param data:数据, len:数据长度
76 * @return
77 * @note 二次开发需要增加自定义命令, 通过JL_OPCODE_CUSTOMER_USER进行扩展, 不要定义这个命令以外的命令, 避免后续SDK更新导致命令冲突
78 */
79 JL_ERR smartbox_user_cmd_send(u8 *data, u16 len)
80 {
81     //自定义数据接收
82     printf("%s:", FUNCTION);
83     put_buf(data, len);
84     return JL_CMD_send(JL_OPCODE_CUSTOMER_USER, data, len, 1);
85 }
86
87 apps/soundbox/smartbox/cmd_data_deal/cmd_user.c
  
```

5.6 功能配置

5.6.1 功能配置

对于所有模式的具体设置的代码都存放在 `soundbox/smartbox/smartbox_setting` 目录下。

```
▼ smartbox_setting/  
    adv_bt_name_setting.c  
    adv_bt_name_setting.h  
    adv_key_setting.c  
    adv_key_setting.h  
    adv_led_setting.c  
    adv_led_setting.h  
    adv_mic_setting.c  
    adv_mic_setting.h  
    adv_time_stamp_setting.c  
    adv_time_stamp_setting.h  
    adv_work_setting.c  
    adv_work_setting.h  
    Makefile  
    smartbox_adv_common.h  
    smartbox_eq_setting.c  
    smartbox_eq_setting.h  
    smartbox_high_low_vol.h  
    smartbox_high_low_vol_setting.c  
    smartbox_music_info_setting.c  
    smartbox_music_info_setting.h  
    smartbox_vol_setting.c  
    smartbox_vol_setting.h
```

对于不需要读写 VM 的功能,可自己新建一个文件实现对应功能,例如 `smartbox_music_info_setting.c`,该文件就是蓝牙模式下实现 ID3 功能的具体设置。如果需要读写 VM,可以按照已有的格式来实现对应的功能。以 `smartbox_high_low_vol_setting.c` 为例子,如需按照这个格式来实现对应的功能,只需填充 `SMARTBOX_SETTING_OPT` 结构体,再调用 `REGISTER_APP_SETTING_OPT` 把结构体放入链表即可。

```
.smartbox_high_low_vol_setting.c .smartbox_setting_opt.h buffers
+ smartbox/
+ browser/
+ bt_manage/
+ cmd_data_deal/
+ func_cmd/
+ smartbox_setting/
+ adv_bt_name_setting.c
+ adv_bt_name_setting.h
+ adv_key_setting.c
+ adv_key_setting.h
+ adv_led_setting.c
+ adv_led_setting.h
+ adv_mic_setting.c
+ adv_mic_setting.h
+ adv_time_stamp_setting.c
+ adv_time_stamp_setting.h
+ adv_work_setting.c
+ adv_work_setting.h
+ Makefile
+ smartbox_adv_common.h
+ smartbox_eq_setting.c
+ smartbox_eq_setting.h
+ smartbox_high_low_vol_setting.c
+ smartbox_high_low_vol_setting.h
+ smartbox_music_info_setting.c
+ smartbox_music_info_setting.h
+ smartbox_vol_setting.c
+ smartbox_vol_setting.h
+ smartbox_setting_opt/
+ smartbox_setting_opt.c
+ smartbox_setting_opt.h
+ config.c
+ event.c
+ feature.c
+ function.c
+ Makefile
+ smartbox.c
+ smartbox_rcsp_manage.c

95 static void deal_high_low_vol(u8 *vol_gain_data, u8 write_vm, u8 tws_sync)
{
    if (vol_gain_data) {
        set_high_low_vol_info(vol_gain_data);
        printf("low %d, high %d\n", high_low_vol.low_vol, high_low_vol.high_vol);
    }
    if (write_vm) {
        update_high_low_vol_vm_value(u8 *)high_low_vol;
    }
    if (tws_sync) {
        high_low_vol_setting_sync(u8 *)high_low_vol;
    }
    high_low_vol_state_update();
}

static SMARTBOX_SETTING_OPT high_low_vol_opt = {
    .data_len = 8,
    .setting_type = ATTR_TYPE_HIGH_LOW_VOL,
    .syscfg_id = CFG_RCSP_ADV_HIGH_LOW_VOL,
    .deal_opt_setting = deal_high_low_vol,
    .set_setting = set_high_low_vol_info,
    .get_setting = get_high_low_vol_info,
    .custom_setting_init = NULL,
    .custom_setting_release = NULL,
    .custom_vm_info_update = NULL,
    .custom_setting_update = NULL,
    .custom_sibling_setting_deal = NULL,
};

95 REGISTER_APP_SETTING_OPT(high_low_vol_opt);

#ifdef SMARTBOX_SETTING_OPT_H
#define SMARTBOX_SETTING_OPT_H
#include "system/includes.h"

6 #define REGISTER_APP_SETTING_OPT(smart_opt) \
int smart_opt##_setting_init(void) { \
    return register_smartbox_setting_opt_setting(&smart_opt); \
} \
static inline void smart_opt##_setting_deinit(void) { \
    return register_smartbox_setting_opt_setting(&smart_opt); \
} \
typedef struct SMARTBOX_SETTING_OPT { \
    struct SMARTBOX_SETTING_OPT *next; \
    u32 data_len; \
    int setting_type; \
    int syscfg_id; \
    void (*deal_opt_setting)(u8 *setting_data, u8 write_vm, u8 tws_sync); \
    void (*set_setting)(u8 *setting_data); \
    void (*get_setting)(u8 *setting_data); \
    int (*custom_setting_init)(void); \
    int (*custom_vm_info_update)(void); \
    int (*custom_setting_update)(u8 *setting_data); \
    int (*custom_sibling_setting_deal)(u8 *setting_data); \
    int (*custom_setting_release)(void); \
} SMARTBOX_SETTING_OPT;

void deal_sibling_setting(u8 *buf);
u8 smartbox_read_data_from_vm(u8 syscfg_id, u8 *buf, u8 buf_len);
int register_smartbox_setting_opt_setting(void *opt_param);

void set_smartbox_opt_setting(u16 setting_type, u8 *data);
void get_smartbox_opt_setting(u16 setting_type, u8 *data);

void smart_setting_init(void);
void update_smartbox_setting(u16 type);
void update_info_from_vm_info(void);
```

如上图所示，SMARTBOX_SETTING_OPT 成员的含义：

data_len：有效数据的长度（必填）

setting_type：功能设置的类型（必填）

syscfg_id：系统配置的类型，用于 VM 的读写（必填）

deal_opt_setting：功能配置具体的实现函数（必填）

set_setting：有效数据的设置（必填）

get_setting：有效数据的获取（必填）

custom_setting_ini：自定义功能设置初始化函数（可选）

custom_setting_release：自定义功能设置释放函数（可选）

custom_vm_info_update：自定义功能设置函数，对端同步后触发，一般需要实现 VM 操作与状态更新即可（可选）

custom_setting_update：自定义设置更新函数，对端同步前数据的获取（可选）

custom_sibling_setting_deal：自定义同步动作，对端同步后数据的填充（可选）

SMARTBOX_SETTING_OPT 结构体的各个成员，都在 smartbox_smetting_opt.c 文件中被调用，具体的实现流程可查看该文件。

```

188 void set_smartbox_opt_setting(u16 setting_type, u8 *data)
{
    SMARTBOX_SETTING_OPT *opt = g_opt_link_head;
    while (opt) {
        if (opt->setting_type == setting_type && opt->deal_opt_setting) {
            opt->deal_opt_setting(data, 1, 1);
        }
        opt = opt->next;
    }
}

void get_smartbox_opt_setting(u16 setting_type, u8 *data)
{
    SMARTBOX_SETTING_OPT *opt = g_opt_link_head;
    while (opt) {
        if (opt->setting_type == setting_type && opt->get_setting) {
            opt->get_setting(data);
        }
        opt = opt->next;
    }
}

void smartbox_opt_release(void)
{
    SMARTBOX_SETTING_OPT *opt = g_opt_link_head;
    while (opt) {
        if (opt->custom_setting_release) {
            opt->custom_setting_release();
        }
        opt = opt->next;
    }
}
    
```

该文件对外提供了三个接口 `set_smartbox_opt_setting`、`get_smartbox_opt_setting` 和 `smartbox_opt_release`，分别对应会调用到结构体中的 `deal_opt_setting`、`get_setting` 和 `custom_setting_release` 三个成员，其中第一个参数就是对应结构体中 `setting_type` 的值，第二个参数是需要操作的数据。

对于 `SMARTBOX_SETTING_OPT` 结构体的 `deal_opt_setting` 成员对应函数的实现一般分为 4 个部分，数据的设置、VM 的写操作、对端同步操作和状态的更新操作，具体请参考 `smartbox_high_low_vol_setting.c` 的 `deal_high_low_vol` 函数。

对于 `SMARTBOX_SETTING_OPT` 结构体的 `setting_type` 成员的值最大不能超过 31。具体使用情况可查看 `soundbox/smartbox/bt_manage/bt_trans_data/le_smartbox_module.h` 文件。

```

#define ATTR_TYPE_BAT_VALUE (0)
#define ATTR_TYPE_EDR_NAME (1)
#define ATTR_TYPE_KEY_SETTING (2)
#define ATTR_TYPE_LED_SETTING (3)
#define ATTR_TYPE_MIC_SETTING (4)
#define ATTR_TYPE_WORK_MODE (5)
#define ATTR_TYPE_PRODUCT_MESSAGE (6)
#define ATTR_TYPE_TIME_STAMP (7)
#define ATTR_TYPE_EQ_SETTING (8)
#define ATTR_TYPE_HIGH_LOW_VOL (9)
#define ATTR_TYPE_VOL_SETTING (10)
    
```

对于 `SMARTBOX_SETTING_OPT` 结构体的 `syscfg_id` 成员具体情况可查看

sounde/include/user_cfg_id.h 文件。

```
// #define VM_RTC_TRIM 18
#define CFG_RCSP_ADV_HIGH_LOW_VOL 19
#define CFG_RCSP_ADV_EQ_MODE_SETTING 20
#define CFG_RCSP_ADV_EQ_DATA_SETTING 21
#define ADV_SEQ_RAND 22
#define CFG_RCSP_ADV_TIME_STAMP 23
#define CFG_RCSP_ADV_WORK_SETTING 24
#define CFG_RCSP_ADV_MIC_SETTING 25
#define CFG_RCSP_ADV_LED_SETTING 26
#define CFG_RCSP_ADV_KEY_SETTING 27
#define CFG_RCSP_ADV_VOL_SETTING 28
// #define CFG_FLASH_BREAKPOINT 29
// #define CFG_USB_BREAKPOINT 30
// #define CFG_SD0_BREAKPOINT 31
// #define CFG_SD1_BREAKPOINT 32
// #define CFG_MUSIC_DEVICE 33
// #define CFG_FM_RECEIVER_INFO 34
// #define CFG_FM_TRANSMIT_INFO 35
// #define CFG_AAP_MODE_INFO 36
// #define CFG_UI_SYS_INFO 37
```

对于 VM 的写可使用 syscfg_write 接口，例如：

```
static void update_high_low_vol_vm_value(u8 *vol_gain_param)
{
    syscfg_write(CFG_RCSP_ADV_HIGH_LOW_VOL, vol_gain_param, sizeof(struct _HIGH_LOW_VOL));
}
```

对于 VM 的读可使用 syscfg_read 或者封装过的 smartbox_read_data_from_vm 接口，例如：

```
u8 smartbox_read_data_from_vm(u8 syscfg_id, u8 *buf, u8 buf_len)
{
    int len = 0;
    u8 i = 0;

    len = syscfg_read(syscfg_id, buf, buf_len);

    if (len > 0) {
        for (i = 0; i < buf_len; i++) {
            if (buf[i] != 0xff) {
                return (buf_len == len);
            }
        }
    }

    return 0;
}
```

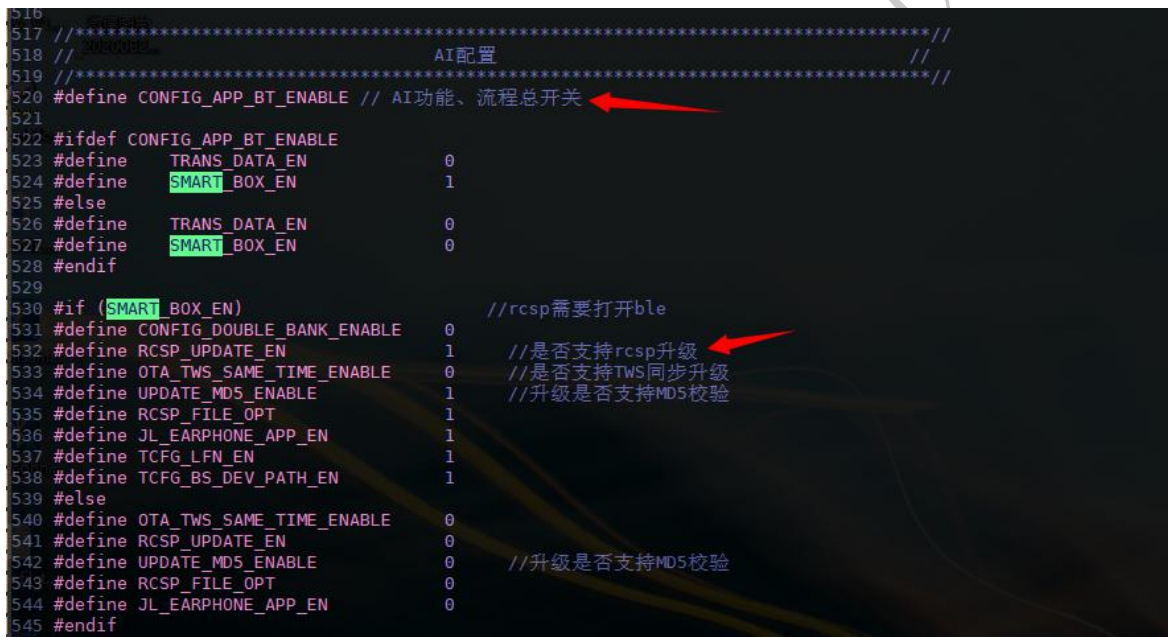
Chapter 6 APP 升级固件说明

6.1 编写目的

该章节主要描述 AC696N 系列芯片 OTA 升级功能，为用户进行二次开发提供参考

6.2 使用 sdk 自带协议进行 OTA 升级

6.2.1 在 board_ac696x_smartbox.h 中打开 CONFIG_APP_BT_ENABLE，如图



```
516
517 //***** AI配置 *****//
518 //***** AI配置 *****//
519 //***** AI配置 *****//
520 #define CONFIG_APP_BT_ENABLE // AI功能、流程总开关
521
522 #ifdef CONFIG_APP_BT_ENABLE
523 #define TRANS_DATA_EN 0
524 #define SMART_BOX_EN 1
525 #else
526 #define TRANS_DATA_EN 0
527 #define SMART_BOX_EN 0
528 #endif
529
530 #if (SMART_BOX_EN) //rcsp需要打开ble
531 #define CONFIG_DOUBLE_BANK_ENABLE 0
532 #define RCSP_UPDATE_EN 1 //是否支持rcsp升级
533 #define OTA_TWS_SAME_TIME_ENABLE 0 //是否支持TWS同步升级
534 #define UPDATE_MD5_ENABLE 1 //升级是否支持MD5校验
535 #define RCSP_FILE_OPT 1
536 #define JL_EARPHONE_APP_EN 1
537 #define TCFG_LFN_EN 1
538 #define TCFG_BS_DEV_PATH_EN 1
539 #else
540 #define OTA_TWS_SAME_TIME_ENABLE 0
541 #define RCSP_UPDATE_EN 0
542 #define UPDATE_MD5_ENABLE 0 //升级是否支持MD5校验
543 #define RCSP_FILE_OPT 0
544 #define JL_EARPHONE_APP_EN 0
545 #endif
```

其中 CONFIG_DOUBLE_BANK_ENABLE 为 1 是使用双备份升级，为 0 是使用单备份升级，在 Flash 空间足够的时候建议使用双备份升级（支持对耳同步进行升级）

6.2.2 从服务器进行升级相关配置

在对应的下载路径下，如 br23\tools\download_ai_double_bank 底下的 json.txt 配置文件中

```
{
  "name": "ver_info",
  "说明": "版本信息:VID (2byte),PID (2byte),VER (2byte)",
  "type": "ARRAY",
  "data": "0x00, 0x02, 0x00, 0x33, 0x00, 0x01"
},
{
  "name": "ver_info_ext",
  "说明": "额外信息 (AuthKey, ProCode)",
  "type": "STRING",
  "data": "hE9yfseX6UdK7rFh,jl_sdk_ac697_publish"
},
}
```

其中 AuthKey 公司申请的用于标志公司信息的 Key, ProCode 为产品代码, VID、PID、VER 根据不同产品和版本号做修改。

6.2.3. MD5 校验说明

如使能 MD5 校验, 则服务器在寻找升级文件时还会判断当前固件的 MD5 值和服务器上的 MD5 列表是否匹配。

```
330 #if (SMART_BOX_EN) //rcsp需要打开ble
331 #define CONFIG_DOUBLE_BANK_ENABLE 0
332 #define RCSP_UPDATE_EN 1 //是否支持rcsp升级
333 #define OTA_TWS_SAME_TIME_ENABLE 0 //是否支持TWS同步升级
334 #define UPDATE_MD5_ENABLE 1 //升级是否支持MD5校验
335 #define RCSP_FILE_OPT 1
336 #define JL_EARPHONE_APP_EN 1
337 #define TCFG_LFN_EN 1
338 #define TCFG_BS_DEV_PATH_EN 1
339 #else
340 #define OTA_TWS_SAME_TIME_ENABLE 0
341 #define RCSP_UPDATE_EN 0
342 #define UPDATE_MD5_ENABLE 0 //升级是否支持MD5校验
343 #define RCSP_FILE_OPT 0
344 #define JL_EARPHONE_APP_EN 0
345 #endif
```

对生成的 ufw 文件名后缀会加上改固件的 md5 值

名称	修改日期	类型	大小
jl_isd.bin	2020/6/22 14:32	BIN 文件	444 KB
jl_isd.fw	2020/6/22 14:32	FW 文件	570 KB
update_01b4e61cf1c16eee991274bccf0cbca3.ufw	2020/6/22 14:32	UFW 文件	1,011 KB
config.dat	2020/6/22 14:32	DAT 文件	1 KB

6.3 客户自带协议进行 OTA 升级

6.3.1 客户自定义升级需要打开宏定义 APP_UPDATE_EN，然后提供升级所需要的 read、seek 等接口注册到升级模块

```
49
50 #if RCSP_UPDATE_EN
51 #define APP_UPDATE_EN
52 #else
53 #define APP_UPDATE_EN
54 #endif
55
```

1 //需要使用APP升级的话要把该宏打开
0 //客户如需要开发自己的app升级协议需要把这个宏打开

6.3.2 相关文件：

```
00 update/ | rcsp_txsf
00:00:1rcsp_ch_loader_download.c
E 00:00:1update_tws.c 16 00 00 42 20 02 00 EF
```

6.3.3、rcsp_ch_loader_download.c 主要实现协议提供给升级模块的接口，客户需要参考该文件根据自己的协议实现相应的接口，注册给升级模块（协议相关流程可以参考 thid_party_profile/JL_rcsp 目录的相关文件）

```
351 void rcsp_update_loader_download_init(int update_type, void (*result_cbk)(void *priv, u8 type, u8 cmd))
352 {
353     #if((OTA_TWS_SAME_TIME_ENABLE && (RCSP_ADV_EN || RCSP_BT_MATE_EN))
354         if(get_tws_sibling_connect_state()) {
355             extern int tws_ota_init(void);
356             tws_ota_init();
357             rcsp_update_loader_downloader_init(
358                 update_type, audio_dac_init
359                 result_cbk,
360                 NULL, &rcsp_update_op);
361             &rcsp_update_op);
362         } else {
363             info: [AUDIO_DAC]DAC Mono LR DIFF OUTPUT
364             [150]info: [AUDIO_DAC]dac trim l .99 r .101 c 0
365             app_update_loader_downloader_init(
366                 update_type,
367                 result_cbk,
368                 NULL,
369                 &rcsp_update_op);
370             info: [APP]enter softpoweroff
371         }
372     }
373 }
```

调用该接口将接口注册给升级模块

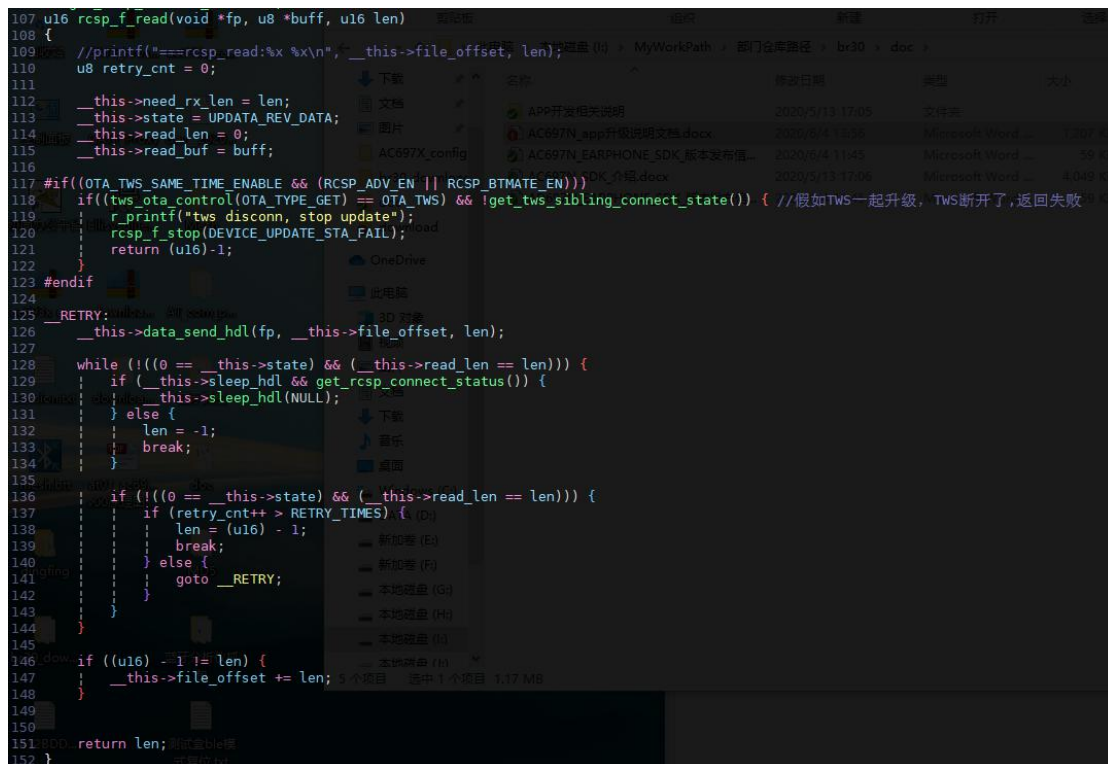
客户需要提供的接口有：

- File_open:

```
153
154 u16 rcsp_f_open(void)
155 {
156     deg_puts(">>>rcsp_f_open\n");
157     __this->file_offset = 0;
158     __this->seek_type = BT_SEEK_SET;
159     return 1;
160 }
161
```

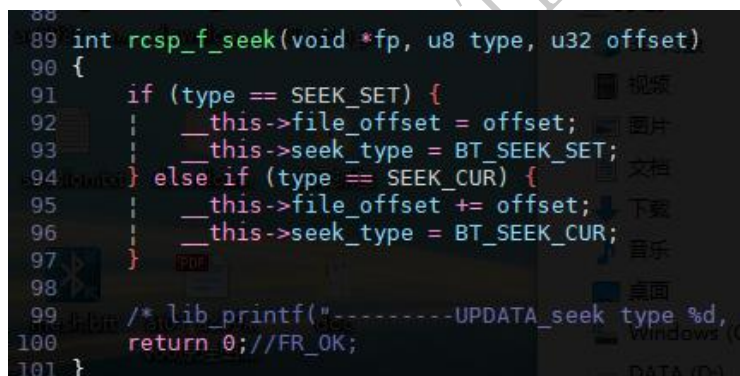
改接口主要为初始化记录当前升级进度的 file_offset 变量。

- File_read:



read 接口实现向主机获取升级数据，调用 data_send_hdl 向主机请求升级文件对应偏移的数据，发送请求后会 Pend 住直到获取到升级数据或者超时返回错误。

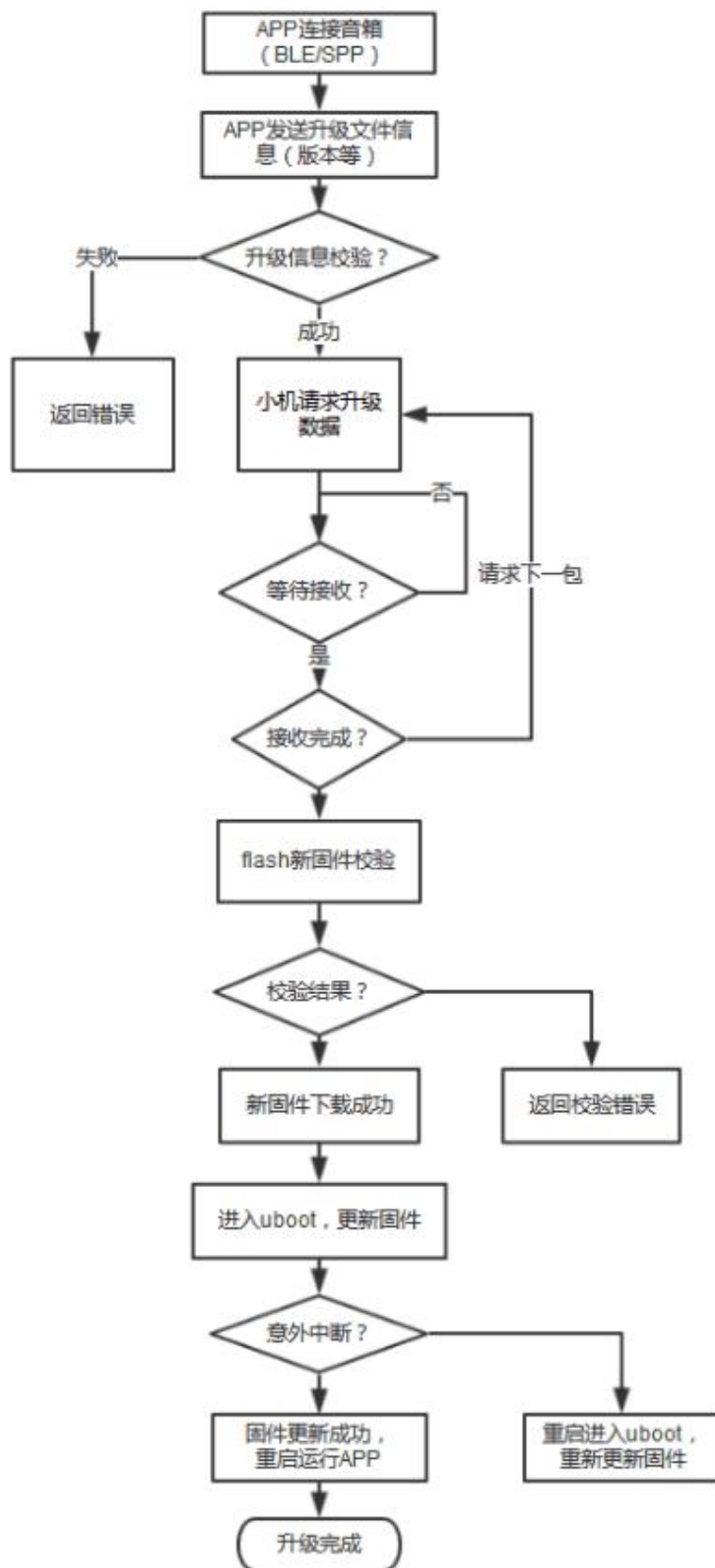
● File_seek:



对 file_offset 参数进行设置

6.3.4、update_tws.c 实现 tws 同步升级的接口和流程，客户不需要修改

6.4 升级流程图



Chapter 7 SD、USB 复用 IO 介绍

7.1 编写目的

该章节主要描述 AC695N、AC696N 系列芯片 sd cmd 复用 linein ad 检测、sd clk 复用 iokey，sd data 与 usb dm 绑定复用的介绍说明。**(sd cmd 复用 linein ad 检测、sd clk 复用 iokey 需要硬件电路支持, 硬件电路的接法不在本文讨论，在制定方案时候请咨询硬件)**

7.2 sd 复用 api 说明

函数原型	u8 sd_io_suspend(u8 sdx, u8 sd_io);
功能描述	强占 sd io 资源
参数说明	Sdx 0:sd0 1:sd1 sd_io 0:cmd 1:clk 2:data
输出	0:强占成功 1: 强占失败，需要 sd 当前正在使用，需要再次强占
例子	<pre>__try_t: if(sd_io_suspend(0, 0)) { os_time_dly(1); printf("wait %s\n", __FUNCTION__); goto __try_t; }</pre>
补充说明	cmd 和 data 使用了同一个控制器，如果有复用 data io 的应用，则不能使用 cmd 检测的方式

函数原型	u8 sd_io_resume(u8 sdx, u8 sd_io)
功能描述	释放强占 sd io 资源
参数说明	Sdx 0:sd0 1:sd1 sd_io 0:cmd 1:clk 2:data
输出	0:释放强占成功 1: 释放强占失败
例子	sd_io_resume(0, 0);
补充说明	强占操作和释放操作必须成对出现，如果有强占没有释放会影响 sd 通信

7.3 sd cmd 复用 linein ad 检测

```

1 //*****
2 //                                linein配置                                //
3 //*****
4 #define TCFG_LINEIN_ENABLE          TCFG_APP_LINEIN_EN // linein使能
5 // #define TCFG_LINEIN_LADC_IDX      0 // linein使用的ladc通道，对应ladc_list
6 #define TCFG_LINEIN_LR_CH          AUDIO_LR0_LR
7 #define TCFG_LINEIN_CHECK_PORT      IO_PORTC_04 // linein检测IO
8 #define TCFG_LINEIN_PORT_UP_ENABLE  1 // 检测IO上拉使能
9 #define TCFG_LINEIN_PORT_DOWN_ENABLE 0 // 检测IO下拉使能
10 #define TCFG_LINEIN_AD_CHANNEL      AD_CH_PC4//NO_CONFIG_PORT // 检测IO是否使用AD检测
11 #define TCFG_LINEIN_VOLTAGE         1000 // AD检测时的阈值
12 #define TCFG_LINEIN_INPUT_WAY       LINEIN_INPUT_WAY_ADC
13 #define TCFG_LINEIN_MULTIPLEX_WITH_SD ENABLE // linein 检测与 SD cmd 复用
14 #define TCFG_LINEIN_SD_PORT         1// 0:sd0 1:sd1 //选择复用的sd

```

一个典型的 linein 复用 sd cmd 配置如上图所示，下面对参与复用的配置项目进行描述。

TCFG_LINEIN_CHECK_PORT：为 ad 的检测 io，需要填写为 sd cmd 的具体 io，根据具体方案来进行填写。

TCFG_LINEIN_AD_CHANNEL : 为 ad 的通道, 头文件在 adc_api.h, 用户可以根据具体方案填写, 本 demo 为 AD_CH_PC4。

TCFG_LINEIN_VOLTAGE : 为 ad 检测的阈值, 用户可以在 linein_sample_mult_sd 函数把 ad 值进行打印出来, 判断阈值。SDK 判断当前 ad 值比阈值低时候, 设备判断为插入。

TCFG_LINEIN_MULTIPLEX_WITH_SD : 为复用的开关。

TCFG_LINEIN_SD_PORT : 为 sd 的 port, 需要根据具体方案进行调整, 例如是 sd0 的 cmd 参与复用, 则此处填写 0, 反之如果是 sd1 的 cmd 参与复用, 则此处是填写 1。

7.4 sd clk 复用 iokey 检测

```
49
50
51 #define TCFG_IO_MULTIPLEX_WITH_SD    ENABLE           // IO_KEY与 SD_CLK复用
52 #define TCFG_MULTIPLEX_PORT         TCFG_IOKEY_PREV_ONE_PORT // 复用的引脚
53
```

如图, 在板卡中增加对应的配置即可支持 sd clk 复用 iokey 的检测, clk io 和按键之间需要串联一个小电阻, 细节的硬件电路的接法不在本文讨论。

TCFG_IO_MULTIPLEX_WITH_SD : 为复用开关。

TCFG_MULTIPLEX_PORT : 为复用的引脚 io, 这里填写具有方案的 clk 引脚。注意事项, 因为具体应用方案可能是 sd0、sd1 的 clk io 参与了复用, 因此在代码中需要注意修改参与复用的 sd。如下图函数在 io_key.c, 的 sd_mult_io_detect 函数

```
133 write ((JL_TIMER0->CON & BIT(15)) == 0),
134 JL_TIMER0->CON = BIT(14);
135 }
136 extern u8 sd_io_suspend(u8 sdx, u8 sd_io);
137 extern u8 sd_io_resume(u8 sdx, u8 sd_io);
138 void sd_mult_io_detect(void *arg)
139 {
140     static u32 cnt = 0;
141     if (sd_io_suspend(1, 1) == 0) {
142         gpio_set_direction(TCFG_MULTIPLEX_PORT, 0);
143         gpio_write(TCFG_MULTIPLEX_PORT, 0);
144         udelay(10);
145         gpio_set_dir(TCFG_MULTIPLEX_PORT, 1);
146         gpio_set_pull_down(TCFG_MULTIPLEX_PORT, 0);
147         gpio_set_pull_up(TCFG_MULTIPLEX_PORT, 1);
148         gpio_set_direction(TCFG_MULTIPLEX_PORT, 1);
149         udelay(10);
150         mult_key_value = gpio_read(TCFG_MULTIPLEX_PORT);
151         sd_io_resume(1, 1);
152     }
153 }
154 #endif
155
```

修改对应sd

7.5 sd data 复用 usb dm

如下图，一个典型的 sd data 复用配置

```
37 //*****//
38 //                                USB 配置                                //
39 //*****//
40 #define TCFG_PC_ENABLE           TCFG_APP_PC_EN//PC模块使能
41 #define TCFG_UDISK_ENABLE       ENABLE_THIS_MODULE//U盘模块使能
42 #define TCFG_OTG_USB_DEV_EN     BIT(0)//USB0 = BIT(0)  USB1 = BIT(1)
43
44 #include "usb_std_class_def.h"
45
46
47 #define TCFG_USB_PORT_CHARGE     DISABLE
48
49 #define TCFG_USB_DM_MULTIPLEX_WITH_SD_DAT0 1
50 #if TCFG_USB_DM_MULTIPLEX_WITH_SD_DAT0
51 //复用情况下，如果使用此USB口作为充电（即LD05V_IN连接到此USB口），
52 //TCFG_OTG_MODE需要或上TCFG_OTG_MODE_CHARGE，用来把charge从host区
53 //分开；否则不需要，如果LD05V_IN与其他IO绑定，则不能或上
54 #define TCFG_DM_MULTIPLEX_WITH_SD_PORT 0//0:sd0 1:sd1 //dm 参与复用的sd配置
55 #undef TCFG_OTG_MODE
56 #define TCFG_OTG_MODE           (TCFG_OTG_MODE_HOST|TCFG_OTG_MODE_SLAVE|TCFG_OTG_MODE_CHARGE|OTG_DET_DP_ONLY)
57
58 #undef USB_DEVICE_CLASS_CONFIG
59 #if TCFG_SD0_SD1_USE_THE_SAME_HW //开启了双卡的可以使能读卡器存储设备
60 #define USB_DEVICE_CLASS_CONFIG (MASSSTORAGE_CLASS|SPEAKER_CLASS|MIC_CLASS|HID_CLASS)
61 #else
62 #define USB_DEVICE_CLASS_CONFIG (SPEAKER_CLASS|MIC_CLASS|HID_CLASS)
63 #endif
64
65 #undef TCFG_SD0_DET_MODE
66 #define TCFG_SD0_DET_MODE       SD_CLK_DECT
67 #define TCFG_USB_SD_MULTIPLEX_IO IO_PORTB_03
68
69 #endif
70
```

下面对参与 usb 和 sd 复用的配置项目进行说明

TCFG_USB_DM_MULTIPLEX_WITH_SD_DAT0：复用的总开关

TCFG_DM_MULTIPLEX_WITH_SD_PORT：参与复用的 sd，根据具体的封装应用场景进行填写。0 为 sd0、1 为 sd1。

TCFG_OTG_MODE：复用情况下，如果使用此 USB 口作为充电（即 LDO5V_IN 连接到此 USB 口），TCFG_OTG_MODE 需要或上 TCFG_OTG_MODE_CHARGE，用来把 charge 从 host 区分开；否则不需要，如果 LDO5V_IN 与其他 IO 绑定，则不能或上

USB_DEVICE_CLASS_CONFIG：从机模式支持的功能配置，如果只有单卡的场景，usb 和 sd 复用下，不支持读卡器功能。**注意事项**：在插入 usb 线场景下播放 sd 卡音乐，sd 卡播放可能有影响，这是无法解决的，干扰来源来着电脑端。用户可以选择性关闭 pc 模式

TCFG_USB_SD_MULTIPLEX_IO：为 sd 卡 data io，用户需要根据具体的应用方案进行填写。

注意事项：如下图 usb 和 sd data 复用场景 sd 不能使用 cmd 检测，只是 sd 控制器决定，因此本 demo 默认重新修改了检测方式为 clk 检测。用户需要根据具体的方案应用选择 clk 检测或者 io 检测。特别注意，如果是 sd1 参与复用，下面的检测方式应该是修改为 sd1 的对应配置。

```
163 #endif
164
165 #undef TCFG_SD0_DET_MODE
166 #define TCFG_SD0_DET_MODE
167 #define TCFG_USB_SD_MULTIPLEX_IO
168
```

```
SD_CLK_DECT
IO_PORTB_03
```

Chapter 8 Audio 模块使用

8.1 编写目的

该文档主要描述 AC700N_SDK audio 模块的使用方法及开发中注意的一些问题，为用户进行二次开发提供参考。

8.2 详细说明

DAC 硬件输出参数配置：

.output:

- 1、DAC_OUTPUT_MONO_L : DACL 单端隔直接地
- 2、DAC_OUTPUT_MONO_R : DACR 单端隔直接地
- 3、DAC_OUTPUT_LR : DACL/R 单端隔直共地
- 4、DAC_OUTPUT_MONO_LR_DIFF : DACL DACR 差分接

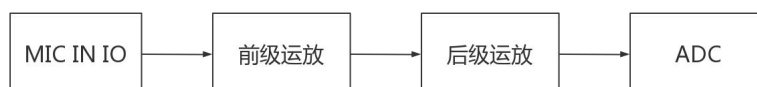
注意：需要根据具体的硬件接法，配置 **TCFG_AUDIO_DAC_CONNECT_MODE** 宏。

另外，在板级文件 DAC 配置结构体中有 VCOMO 使能配置，VCOMO 可以输出与 DAC 引脚同样的直流，当使能 VCOMO，可以将 VCOMO 作为单端输出的公共地，可以省去隔直电容。

```
/* ***** DAC ***** */
#if TCFG_AUDIO_DAC_ENABLE
struct dac_platform_data dac_data = {
    .ldo_volt      = TCFG_AUDIO_DAC_LDO_VOLT,           //DACVDD等级,需要根据具体硬件来设置(高低压)可选:1.2V/1.3V/2.35V/2.5V/2.65V/2.8V/2.95V/3.1V
    .vcmo_en       = 1,                                //是否打开VCOMO
    .output        = TCFG_AUDIO_DAC_CONNECT_MODE,       //DAC输出配置,和具体硬件连接有关,需根据硬件来设置
    .ldo_isel      = 3,
    .ldo_fb_isel   = 3,
    .lpf_isel      = 0x8,
    .zero_cross_detect = 1,
};
#endif
```

8.3 MIC 和 LINEIN 增益调节

音频信号从芯片引脚输入，经过运放后输出到 ADC 采集转换成数字信号。MIC 和 LINEIN 的信号输入最后都是同样的 ADC 信号采集流程，但是根据 MIC 和 LINEIN 的使用场景不同，运放的增益档位设计是不同的。MIC 和 LINEIN 的信号输入流程图如下：



MIC 输入通路流程图



LINEIN 输入通路流程图

一般来说，MIC 器件输出的信号电压幅度小，所以 MIC 的通路有两级运放，经过两级运放的放大作用再给 ADC 采集。

AC696N 的 LINEIN 和 MIC 通路 ADC 端电压采集幅度范围都是 640mVpp 以内，而由于 IO 端的直流偏置是 0.6V，所以 IO 端限制的电压输入范围是 1.2Vpp。调节 LINEIN 和 MIC 的增益，实际上是调整整个通路运放的增益倍数。

例如 LINEIN 通路要获得最大输入幅度 1.2Vpp，当输入 1.2Vpp 的时候需要 ADC 模块的数值输出范围接近 0dB，则最佳的增益选择应是 $20 \cdot \log(640/1200) \approx -5.5\text{dB}$ 。这时应选择增益挡位 3(-5dB)或 2(-6dB)，当增益选择挡位 3 时，输入信号可能幅度在未到达 1.2Vpp 时，ADC 输出数值就会接近 0dB，输入信号再大则会失真。当增益选择挡位 2 时，输入信号最大输入为 1.2Vpp(因为直流偏置 0.6V 限制了)，此时 ADC 输出数值会小于 0dB。

AC696N AUDIO ADC 增益挡位具体对应倍数如下：

MIC 前级运放：

0-1 一共 2 级，每级增益 step 是 6dB。

函数接口：void audio_mic_0dB_en(bool en); // 0:6dB 1:0dB

MIC 后级运放：

0-14 一共 15 级，每级增益 step 是 2dB。

0:0dB 1:2dB 3:4dB ... 14:28dB

函数接口：int audio_adc_mic_set_gain(struct adc_mic_ch *mic, int gain);

LINEIN 运放：

0-15 一共 16 级，每级增益 step 是 1dB。

0:-8dB 1:-7dB 2:-6dB ... 15:7dB

函数接口：int audio_adc_linein_set_gain(struct audio_adc_ch *ch, int gain);

Chapter 9 客户常见问题反馈解答

Q1:

Q2:
